LECTURE 03 ANN

Gradient Descent (GD)

- Uses **all training data** to compute the gradient of the loss function.
- Update step:

$$w \leftarrow w - \eta \nabla L(w)$$

  where $L(w)$ is the total loss over the full dataset.
- **Pros:** Stable and smooth convergence.
- **Cons:** Very slow if dataset is large (since each update scans all data).

Stochastic Gradient Descent (SGD)

- Uses **only one random training example** (or a very small subset) to compute the gradient.
- Update step:

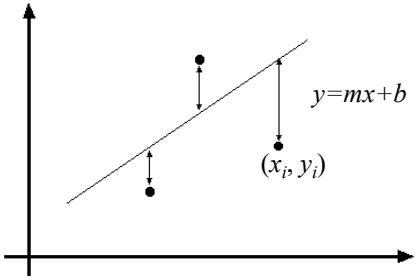
$$w \leftarrow w - \eta \nabla L_i(w)$$

  where $L_i(w)$ is the loss for a single training sample.
- **Pros:** Much faster per update, works well for very large datasets, can escape local minima.
- **Cons:** Noisy updates, convergence path "zig-za ↓ instead of being smooth.

Linear Regression: Least Square Error Solution

Model

- Data: $(x_1, y_1), \ldots, (x_n, y_n)$
- Line equation: $y_i = m x_i + b$
- Find (m, b) to minimize



Error Function

$$E = \sum_{i=1}^{n} (y_i - m x_i - b)^2$$

$$E = \sum_{i=1}^{n} \left(\begin{bmatrix} x_i & 1 \end{bmatrix} \begin{bmatrix} m \\ b \end{bmatrix} - y_i \right)^2 = \left\| \begin{bmatrix} x_1 & 1 \\ \vdots & \vdots \\ x_n & 1 \end{bmatrix} \begin{bmatrix} m \\ b \end{bmatrix} - \begin{bmatrix} y_1 \\ \vdots \\ y_n \end{bmatrix} \right\|^2 = \|\mathbf{Ap} - \mathbf{y}\|^2$$

$$= \mathbf{y}^T\mathbf{y} - 2(\mathbf{Ap})^T\mathbf{y} + (\mathbf{Ap})^T(\mathbf{Ap})$$

$$\frac{dE}{dp} = 2\mathbf{A}^T\mathbf{Ap} - 2\mathbf{A}^T\mathbf{y} = 0$$

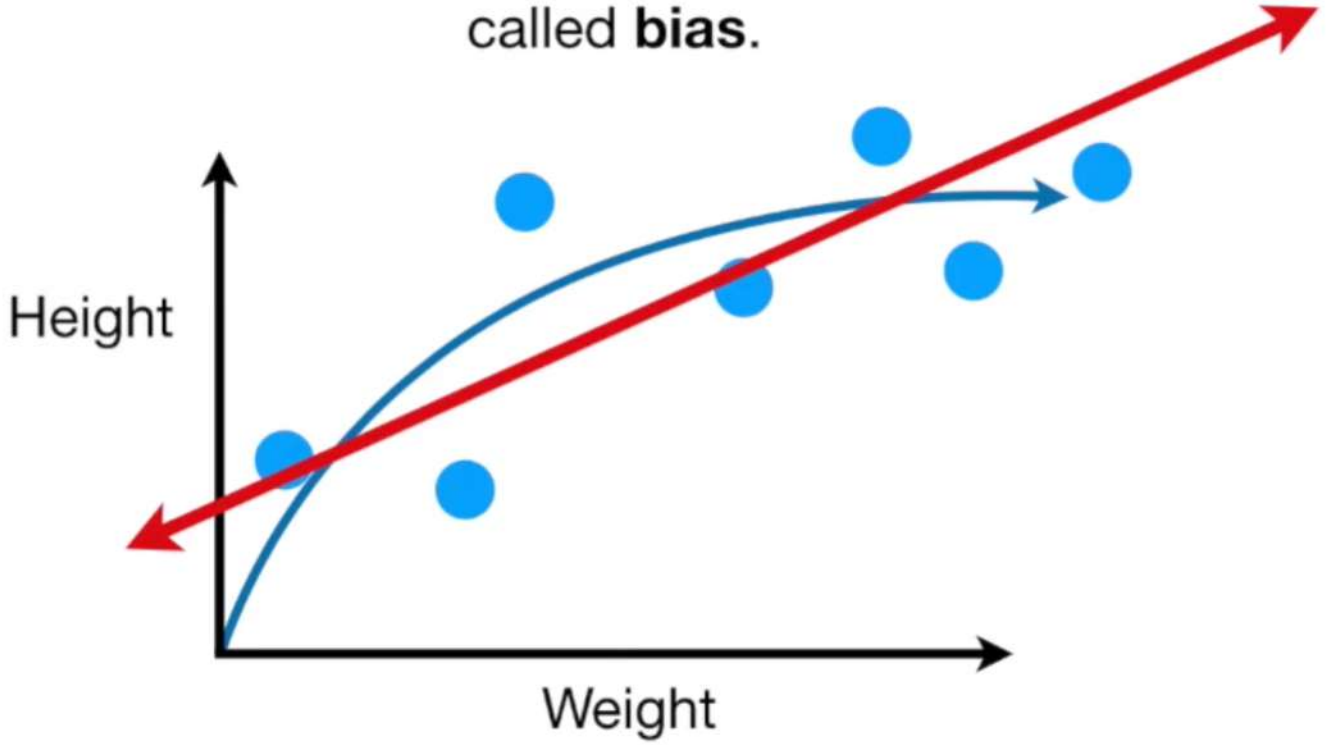
$$\mathbf{A}^T\mathbf{Ap} = \mathbf{A}^T\mathbf{y} \Rightarrow \mathbf{p} = \left(\mathbf{A}^T\mathbf{A}\right)^{-1}\mathbf{A}^T\mathbf{y}$$

Matlab: `p = A \ y;`

Modified from S. Lazebnik

Bais , Variance, Overfitting, Underfitting

The inability for a machine learning method (like linear regression) to capture the true relationship is called **bias**.



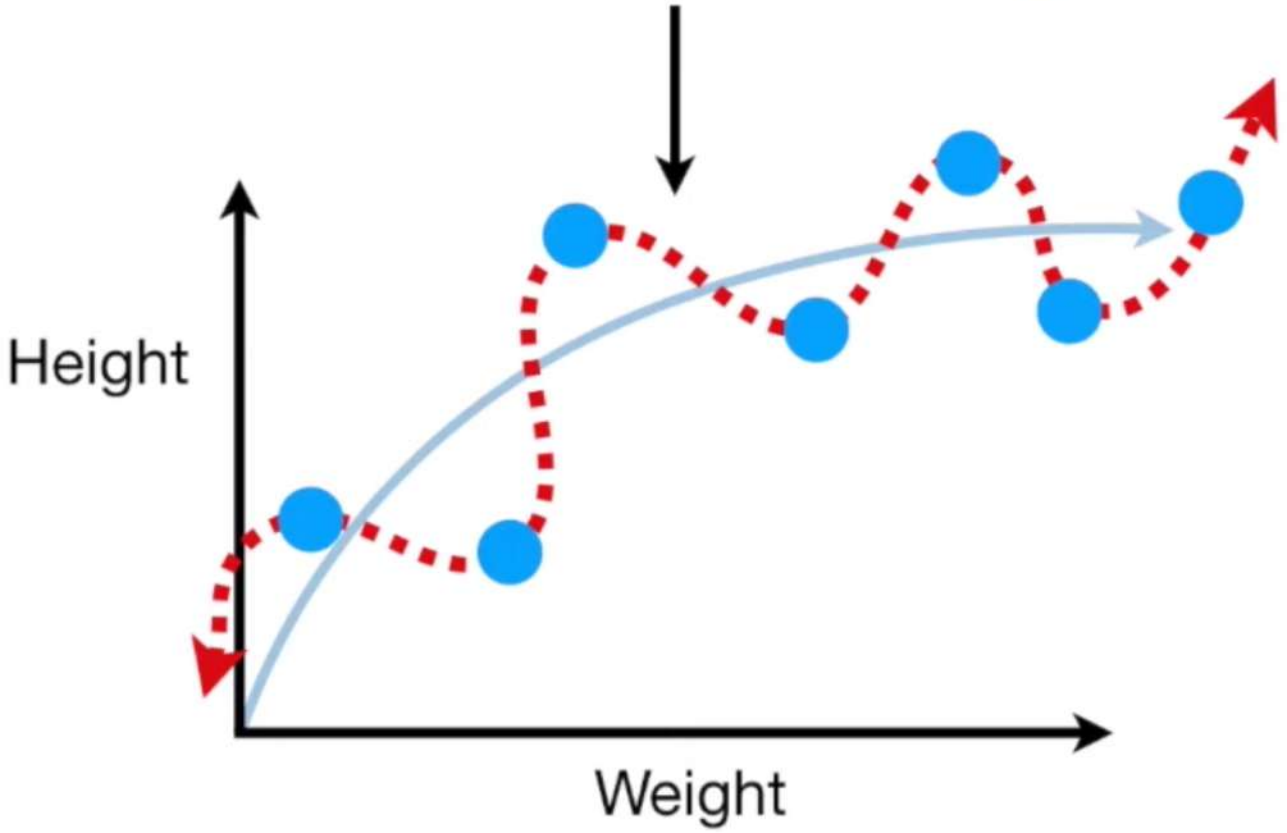
2. Bias

- **Definition:** Bias measures how much the *average prediction* of our model differs from the true function $f(x)$.
- **Formula:**

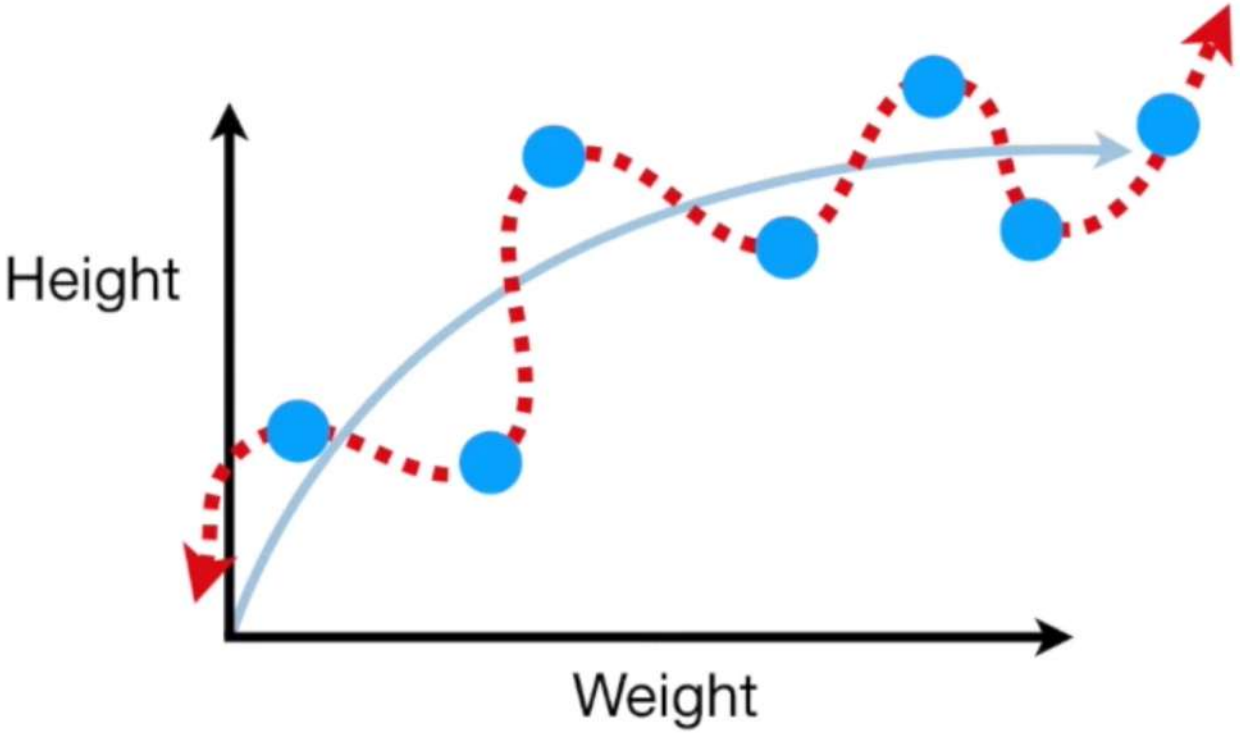
- **Intuition:**
  - If our linear model is too simple (e.g., the true relation is quadratic but we fit a line), our model will **systematically miss** the true pattern → **high bias**.
  - Bias reflects **underfitting**.

Example:
True relation $f(x) = x^2$, but we fit $\hat{y} = mx + c$. The line can't capture the curve, so predictions are consistently off → large bias.

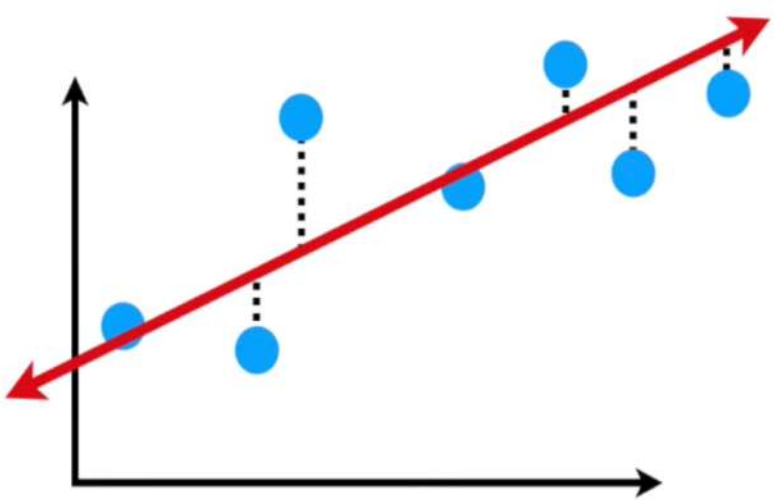
Another machine learning method might fit a **Squiggly Line** to the **training set**…

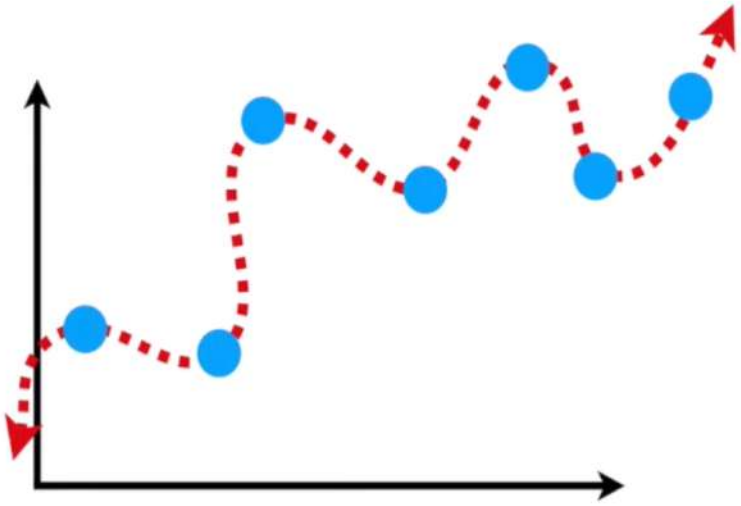


Because the **Squiggly Line** can handle the arc in the true relationship between weight and height, it has very little **bias**.

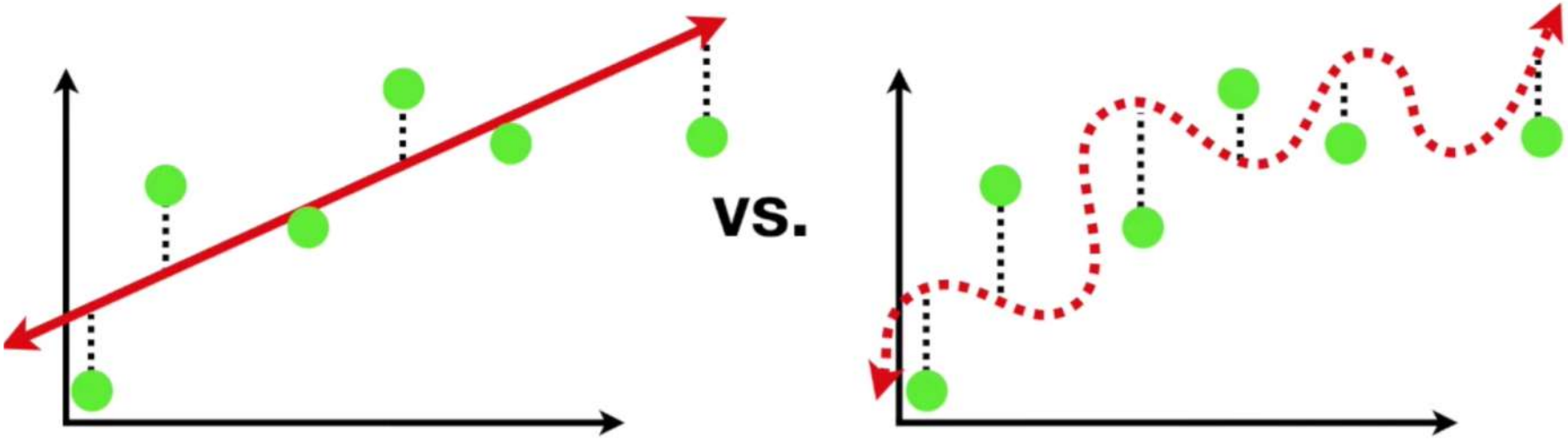


We can compare how well the **Straight Line** and the **Squiggly Line** fit the **training set** by calculating their sums of squares.

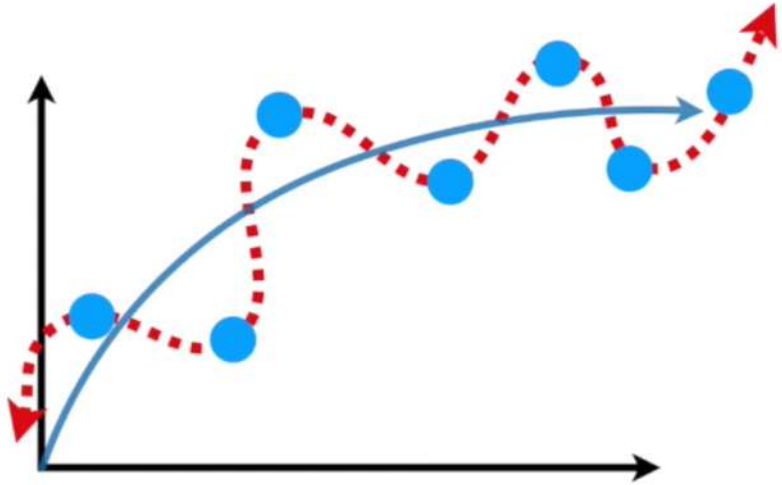


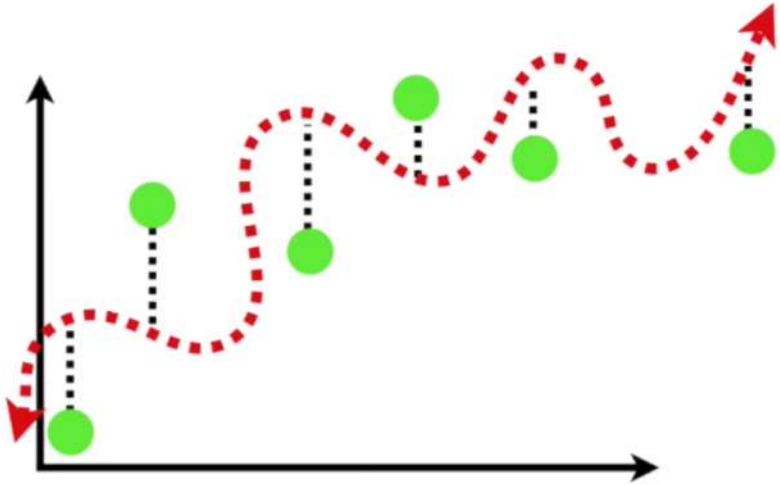


In the contest to see whether the **Straight Line** fits the **testing set** better than the **Squiggly Line**…



Because the **Squiggly Line** fits the **training set** really well, but not the **testing set**, we say that the **Squiggly Line** is **overfit**.





3. Variance

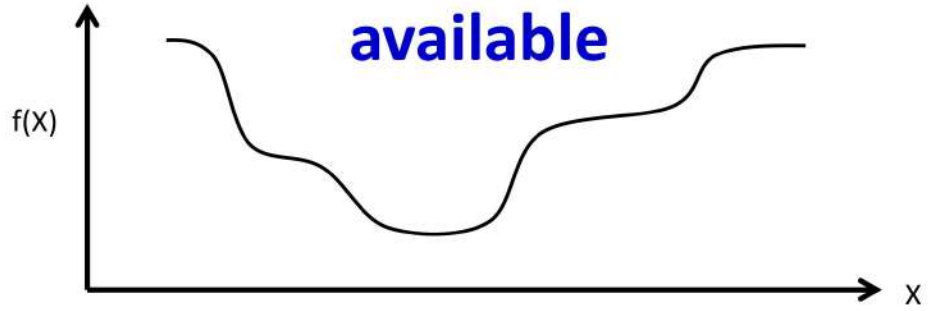
- **Definition:** Variance measures how much predictions $\hat{y}(x)$ would vary if we trained on **different random samples** of the data.
- **Formula:**

- **Intuition:**
  - If our model changes drastically with small changes in training data, variance is high.
  - High-degree polynomials, for example, can wiggle a lot between datasets → **overfitting**.

Example:

Fit a 10th-degree polynomial to 10 noisy points → predictions vary wildly with new samples → high variance.

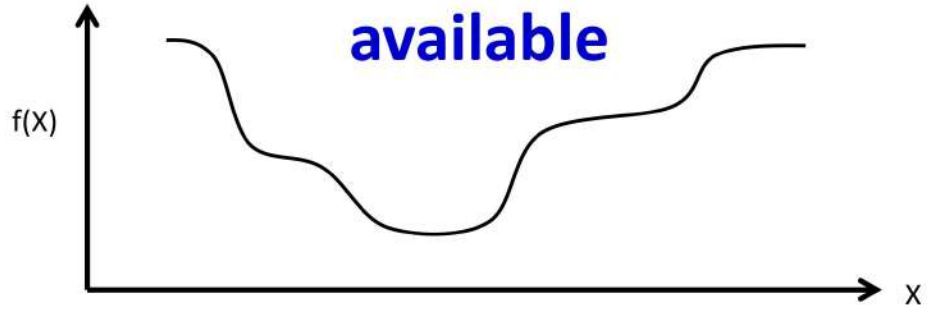
Closed Form Solutions are not always available



- Often it is not possible to simply solve $\nabla_X f(X) = 0$
  - The function to minimize/maximize may have an intractable form

Assume we have 100K images each images of size 100 by 100.
This means that our X is 100k by 10K.
$X^T X$ will be computationally very expensive

Closed Form Solutions are not always available



- Often it is not possible to simply solve $\nabla_X f(X) = 0$
  - The function to minimize/maximize may have an intractable form
- In these situations, iterative solutions are used
  - Begin with a "guess" for the optimal X and refine it iteratively until the correct value is obtained

Stanford
ONLINE

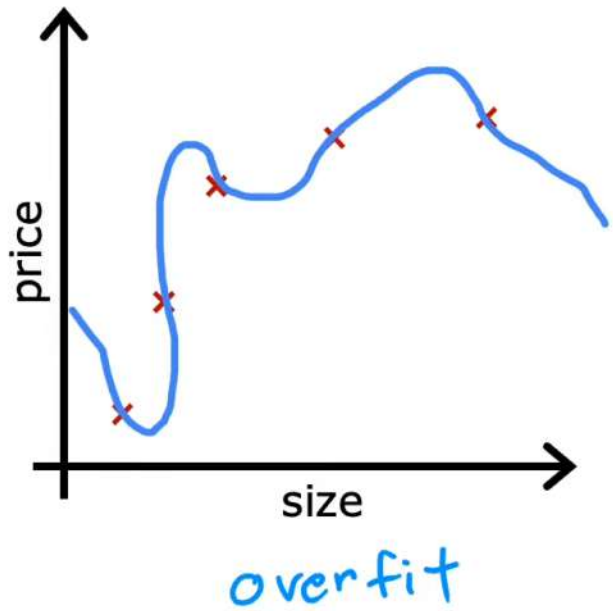
DeepLearning.AI

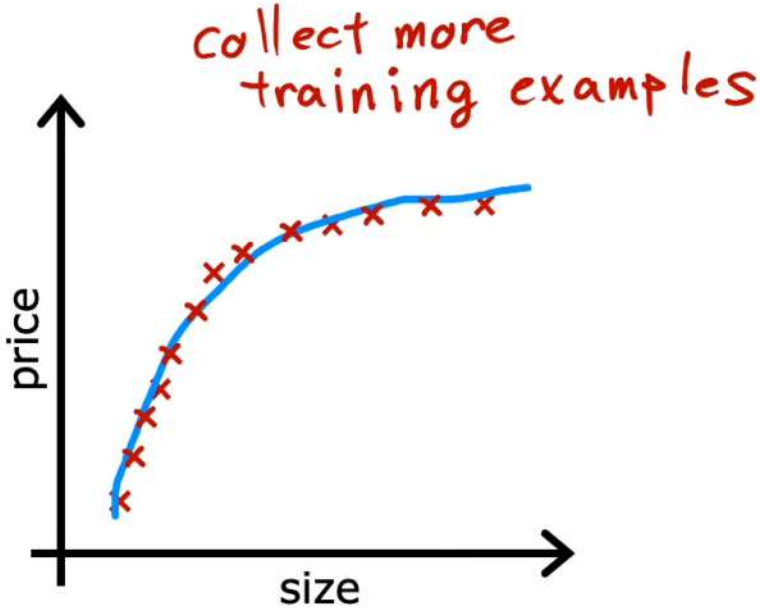


Regularization to Reduce Overfitting

Addressing Overfitting

Collect more training examples





Select features to include/exclude



all features

+

insufficient data

↓

overfit

selected features

size
bedrooms
age
just right

feature selection

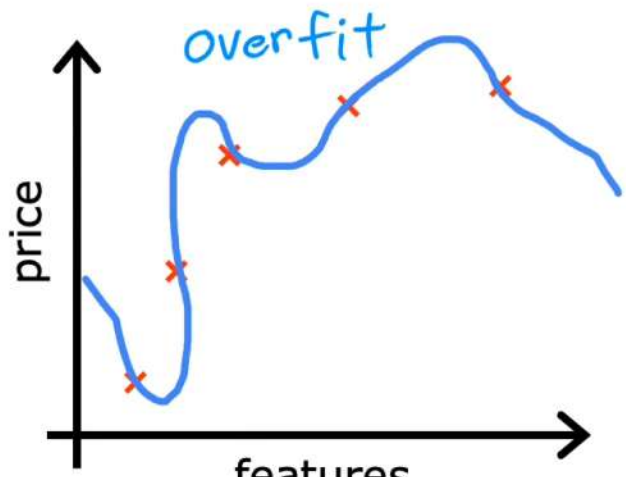
disadvantage

↓

useful features could be lost

Regularization

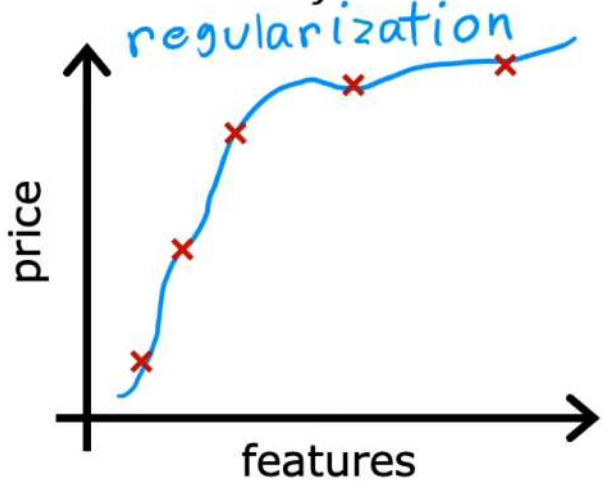
Reduce the size of parameters w_j



$$f(x) = 28x - 385x^2 + 39x^3 - 174x^4 + 100$$

large values for w_j

eliminate feature



$$f(x) = 13x - 0.23x^2 + 0.000014x^3 - 0.0001x^4 + 10$$

small values for w_j

Reducing the **magnitude of weights** helps avoid overfitting because it makes the neural network **less sensitive to individual training examples** and encourages it to learn simpler patterns.

1. Large weights → complex, sensitive model

Suppose a neuron computes:

$$z = w_1 x_1 + w_2 x_2 + b$$

If the weights are very large, a small change in the input can cause a large change in z.

For example:

$$z = 10x_1 + 15x_2$$

The model can make very sharp decision boundaries and potentially memorize peculiarities/noise in the training data.

Large weights → high sensitivity → more complex decision boundary → higher risk of overfitting.

2. Small weights → smoother model

Now suppose we constrain the weights:

$$z = 0.5x_1 + 0.8x_2$$

The output changes more gradually when the input changes.

This tends to produce a **smoother and simpler decision function**, which is more likely to capture general patterns rather than noise.

Small weights → lower sensitivity → simpler model → better generalization.

1. L2 Regularization (Ridge)

We add a penalty term proportional to the **square of weights**:

$$L_{\text{reg}} = L + \lambda \sum_i w_i^2$$

- Encourages weights to be small but spread out.
- Common in neural nets as **weight decay**.

Update rule looks like:

$$w_{t+1} = (1 - \eta\lambda)w_t - \eta \nabla L(w_t)$$

👉 Each step shrinks weights a bit (decay).

---

2. L1 Regularization (Lasso)

We add a penalty proportional to the **absolute value of weights**:

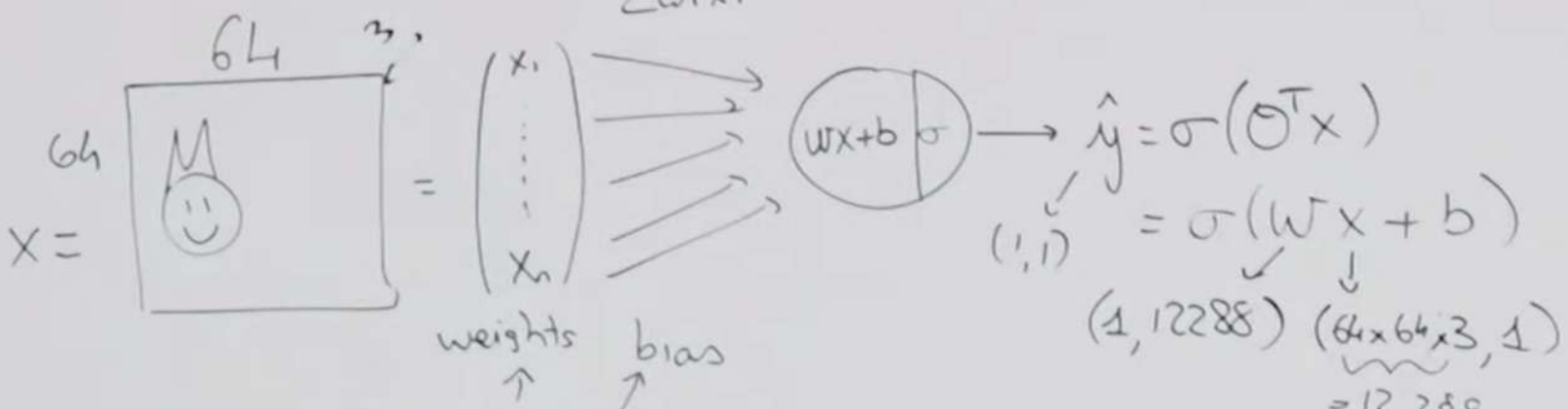
$$L_{\text{reg}} = L + \lambda \sum_i |w_i|$$

- Encourages weights to go exactly to zero.
- Useful for **sparse models** (feature selection).

(1) Logistic Regression

goal Find cats in images { 1 → presence of a cat
{ 0 → absence of a cat

$\sigma' = \sigma(1-\sigma)$



i) initialize w, b

ii) Find the optimal w, b → $\mathcal{L} = -\left[y \log \hat{y} + (1-y) \log(1-\hat{y})\right]$

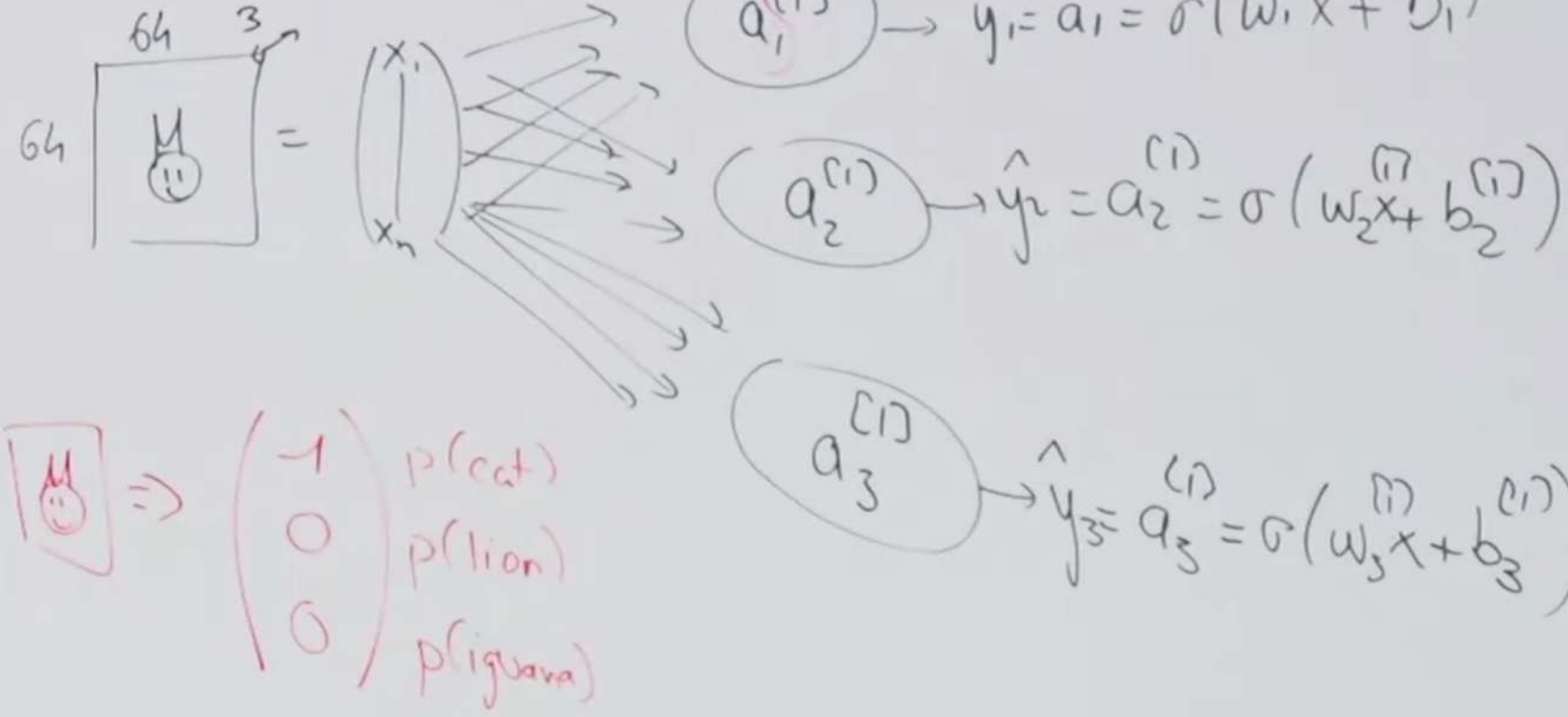
iii) Use $\hat{y} = \sigma(wx+b)$ to predict

$$\begin{cases} w = w - \alpha \frac{\partial \mathcal{L}}{\partial w} \\ b = b - \alpha \frac{\partial \mathcal{L}}{\partial b} \end{cases}$$

(Eq1) neuron = linear + activation

(Eq2) model = architecture + parameters

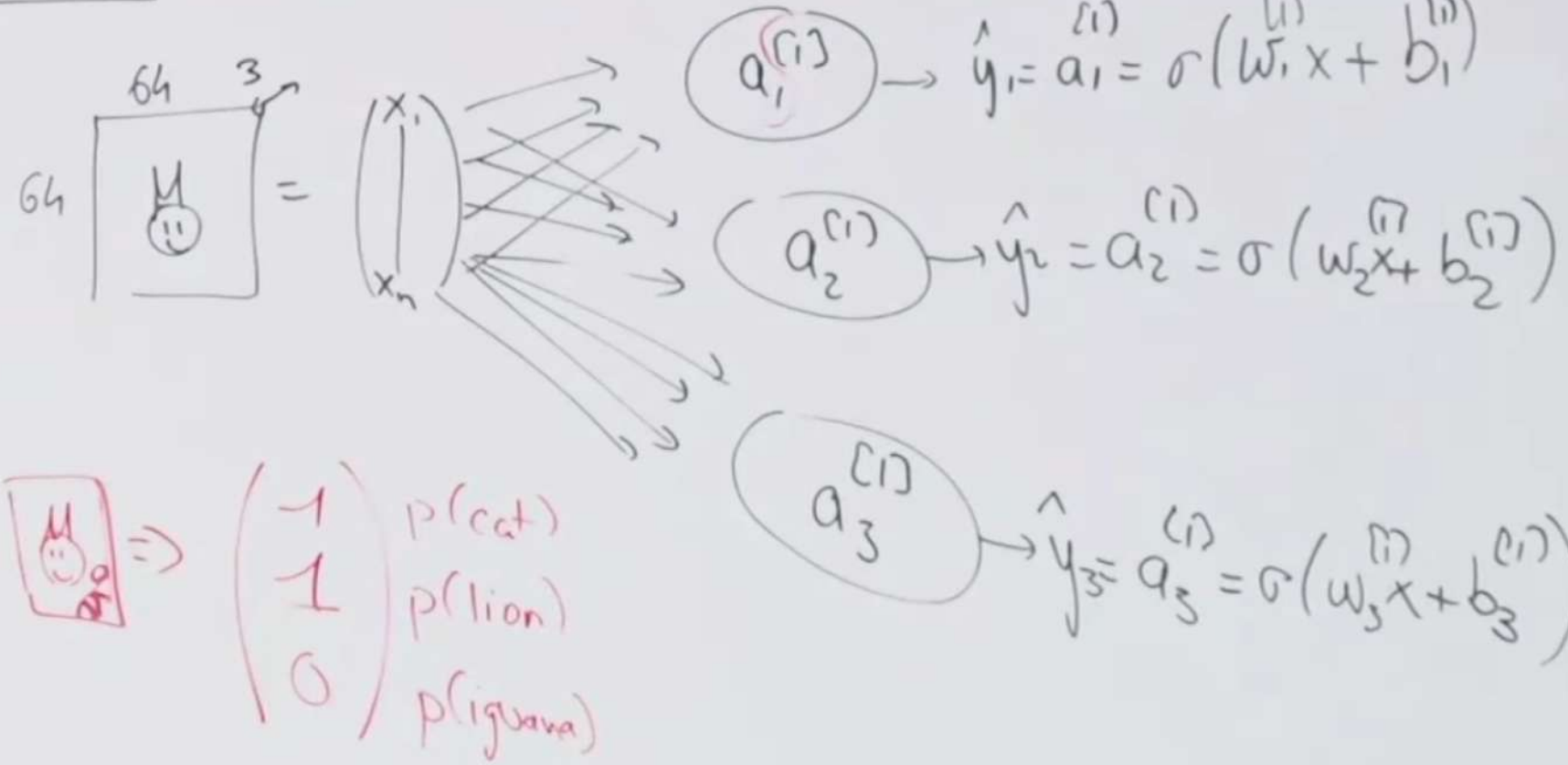
goal 2.0 : Find cat / lion / iguana in images.



(Eq1) neuron = linear + activation

(Eq2) model = architecture + parameters

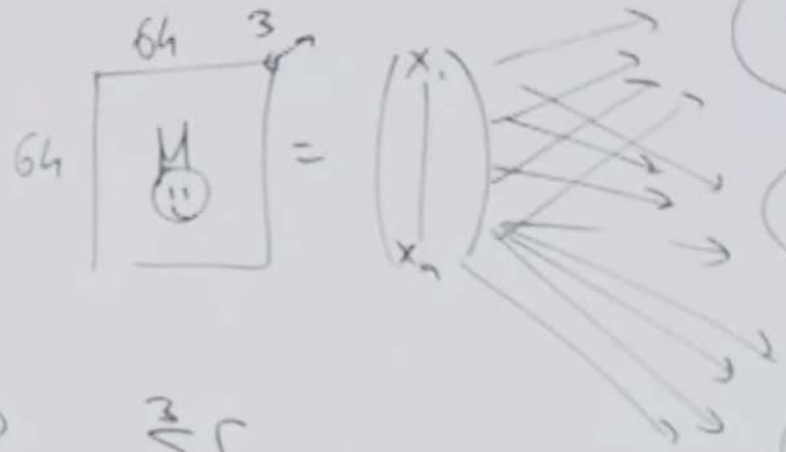
goal 2.0 : Find cat / lion / iguana in images.



(Eq1) neuron = linear + activation

(Eq2) model = architecture + parameters

goal 2.0 Find cat / lion / iguana in images



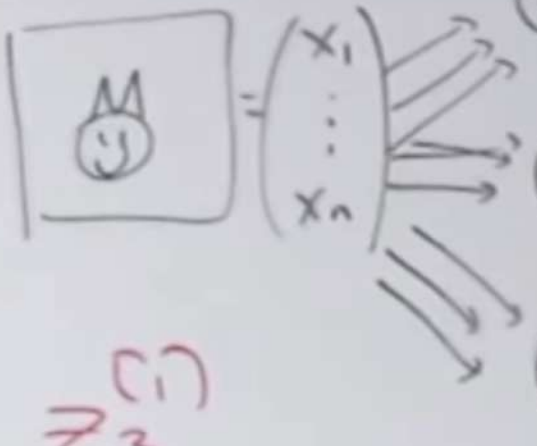
$a_1^{[1]} \rightarrow \hat{y}_1 = a_1^{[1]} = \sigma(w_1^{[1]} x + b_1^{[1]})$

$a_2^{[1]} \rightarrow \hat{y}_2 = a_2^{[1]} = \sigma(w_2^{[1]} x + b_2^{[1]})$

$a_3^{[1]} \rightarrow \hat{y}_3 = a_3^{[1]} = \sigma(w_3^{[1]} x + b_3^{[1]})$

$$\mathcal{L}_{sv} = -\sum_{k=1}^{3} \left[y_k \log \hat{y}_k + (1 - y_k) \log (1 - \hat{y}_k) \right]$$

goal 3.0 : + constraint
unique animal on an image



$$z_1^{(i)} = e^{z_1^{(i)}} \Big/ \sum_{k=1}^{3} e^{z_k^{(i)}}$$

$$z_2^{(i)} = e^{z_2^{(i)}} \Big/ \sum_{k=1}^{3} e^{z_k^{(i)}}$$

$$z_3^{(i)} = e^{z_3^{(i)}} \Big/ \sum_{k=1}^{3} e^{z_k^{(i)}}$$

$$e^{z_3^{(i)}} \Big/ \sum_{k=1}^{3} e^{z_k^{(i)}}$$

Softmax

$$\begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix} \quad \begin{pmatrix} 0 \\ 1 \\ 0 \end{pmatrix} \quad \begin{pmatrix} 0 \\ 0 \\ 1 \end{pmatrix}$$

multi-class regression

🔷 Case 1: Single-label classification

- Example: ImageNet classification ("what's in this image?").
- Even if the image has many things, the dataset assumes **only one ground-truth label** (e.g., "dog" not "dog + cat").
- Here, the **softmax** makes sense because it forces the model to pick **one class with the highest probability**.

👉 Softmax ensures:

$$\sum_{i=1}^{K} p_i = 1, \quad 0 \leq p_i \leq 1$$

so the model commits to one class.

🔷 Case 2: Multi-class, multi-object detection

- Example: Object detection with **DETR, Faster R-CNN, YOLO**.
- Each **bounding box** has its own classifier head with **softmax over object categories + background**.
- That way, every region/box chooses the *most likely* object class.

👉 Softmax is used **per object candidate**, not across the whole image.

🔷 Case 3: Multi-label classification (not detection)

- Example: "Does this image contain a dog, a cat, and/or a car?" (multiple can be true).
- Here **softmax is not used**, because softmax assumes exclusivity.
- Instead, we use **sigmoid per class** → each label gets an independent probability.

✅ So how does softmax help with multiple objects?

- In **object detection**, softmax is applied **locally per region/box**, not globally over the entire image.
- This allows the model to say: "Box 1 is a dog (p=0.95), Box 2 is a cat (p=0.88), Box 3 is background."
- Without softmax, it would be harder to normalize class predictions for each object proposal.

✅ So, the trick is:

- In **classification**, softmax = single label only.
- In **object detection**, softmax is applied per bounding box/query, so multiple objects can be detected simultaneously.

1. Cross-Entropy Loss (a.k.a. Categorical Cross-Entropy)

- **Used for:** Multi-class classification
- **Setup:** You have **one true class** out of many.
- **Target (y):** One-hot vector (e.g., for 3 classes → `[0, 1, 0]`).
- **Model output (p):** Softmax probabilities over all classes (sum = 1).

Formula:

$$L = -\sum_{c=1}^{C} y_c \log(p_c)$$

where C = number of classes.

Example:

- Classes = {cat, dog, horse}
- True = "dog" → `[0,1,0]`
- Model = `[0.2, 0.7, 0.1]`
- Loss = $-\log(0.7)$.

🔷 Example

Suppose we classify an image among 3 classes: **Cat, Dog, Rabbit**

- True label: **Cat** → one-hot = [1, 0, 0]
- Model prediction (softmax output): [0.7, 0.2, 0.1]

Loss:

$$L = -(1 \cdot \log(0.7) + 0 \cdot \log(0.2) + 0 \cdot \log(0.1)) = -\log(0.7) \approx 0.356$$

If prediction was worse, say [0.2, 0.5, 0.3], then

$$L = -\log(0.2) = 1.609$$

(higher loss, meaning worse prediction).

2. Binary Cross-Entropy (BCE) Loss

- **Used for:** Binary classification (yes/no, 0/1) **or multi-label classification** (independent classes).
- **Target (y):** Scalar (0 or 1) for binary, or vector of 0/1 for multi-label.
- **Model output (p):** Sigmoid probabilities for each class **independently** (not normalized across classes).

Formula (single class):

$$L = -\left[y \log(p) + (1 - y) \log(1 - p)\right]$$

Formula (multi-label, K classes):

$$L = -\sum_{k=1}^{K} \left[y_k \log(p_k) + (1 - y_k) \log(1 - p_k)\right]$$

Example (binary):

- True = 1 (positive)
- Model = 0.8
- Loss = $-\log(0.8)$.

Example (multi-label):

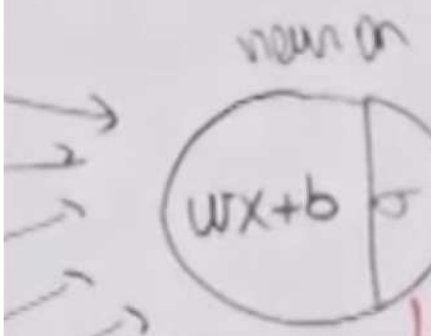
- Labels = {cat ✅, dog ❌, horse ✅} → `[1,0,1]`
- Model = `[0.9, 0.3, 0.6]`
- Loss = BCE applied **independently** to each label

🔑 Key Difference

- **Cross-Entropy:**
  - Exactly **one class is true**.
  - Uses **softmax** → forces probabilities to compete and sum to 1.
  - Good for *multi-class problems*.
- **Binary Cross-Entropy:**
  - Each class is **independent (0 or 1)**.
  - Uses **sigmoid** → independent probabilities.
  - Good for *binary or multi-label problems*.

👉 In short:

- **Softmax + Cross-Entropy** = one label per input.
- **Sigmoid + BCE** = multiple independent labels possible.



$\hat{y} = \sigma(\theta^T x) = \sum \theta_i x_i + \theta_0 x_0$

$= \sigma(w^T x + b)$

$(1,1)$

$(1, 12288)$ $(64 \times 64 \times 3, 1)$

$= 12,288$

ReLU

$\mathcal{L} = -\left[y \log \hat{y} + (1-y) \log (1-\hat{y}) \right]$

$\begin{cases} w = w - \alpha \frac{\partial \mathcal{L}}{\partial w} \\ b = b - \alpha \frac{\partial \mathcal{L}}{\partial b} \end{cases}$

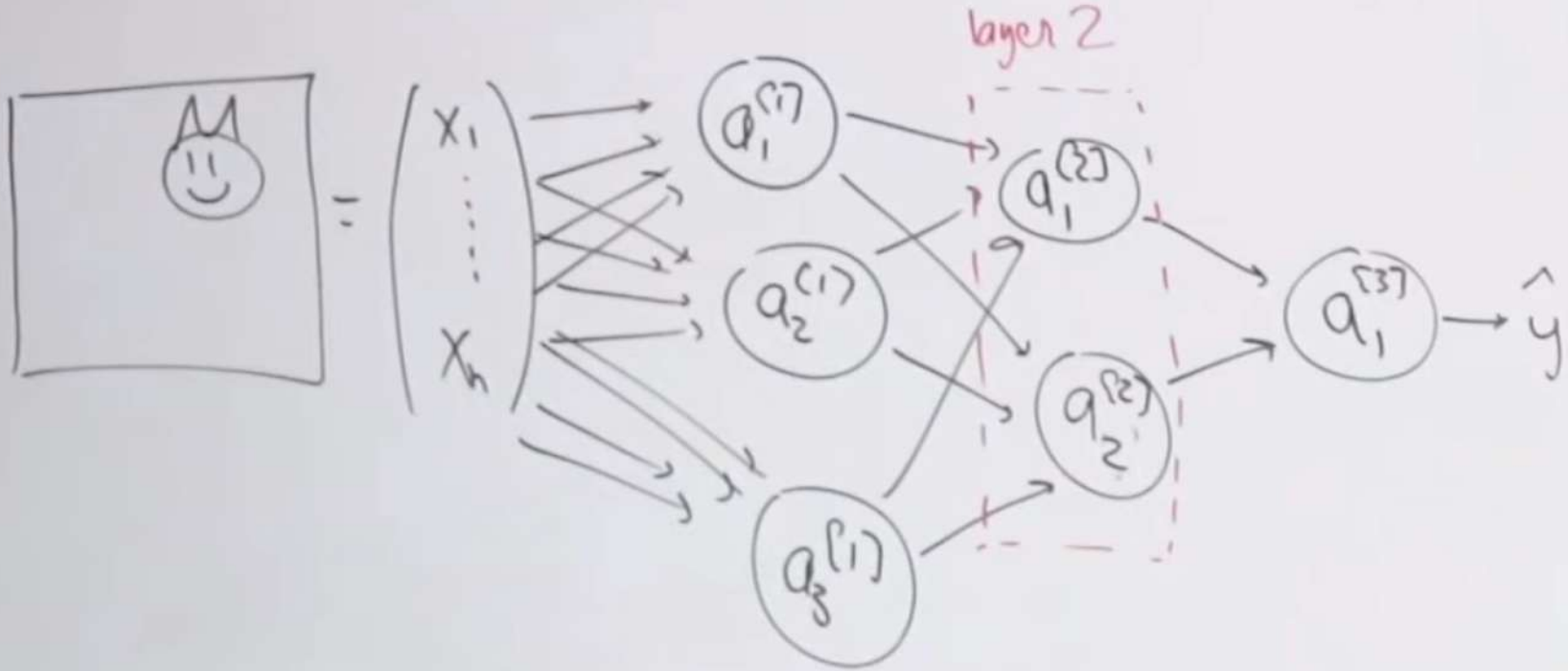
$|\hat{y} - y|$

,b

o predict

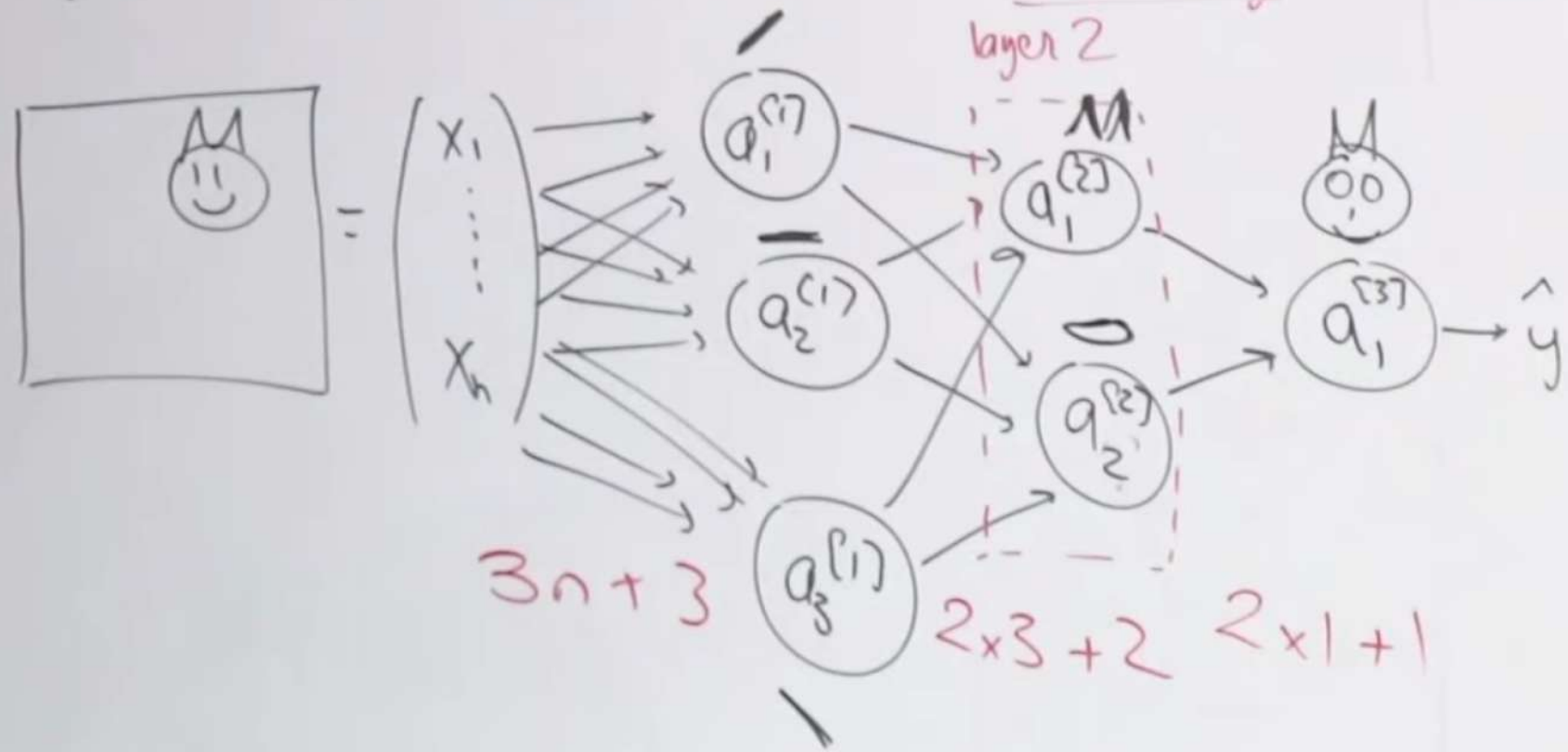
II Neural Networks

goal image ⇒ cat (1) vs. no cat (0)

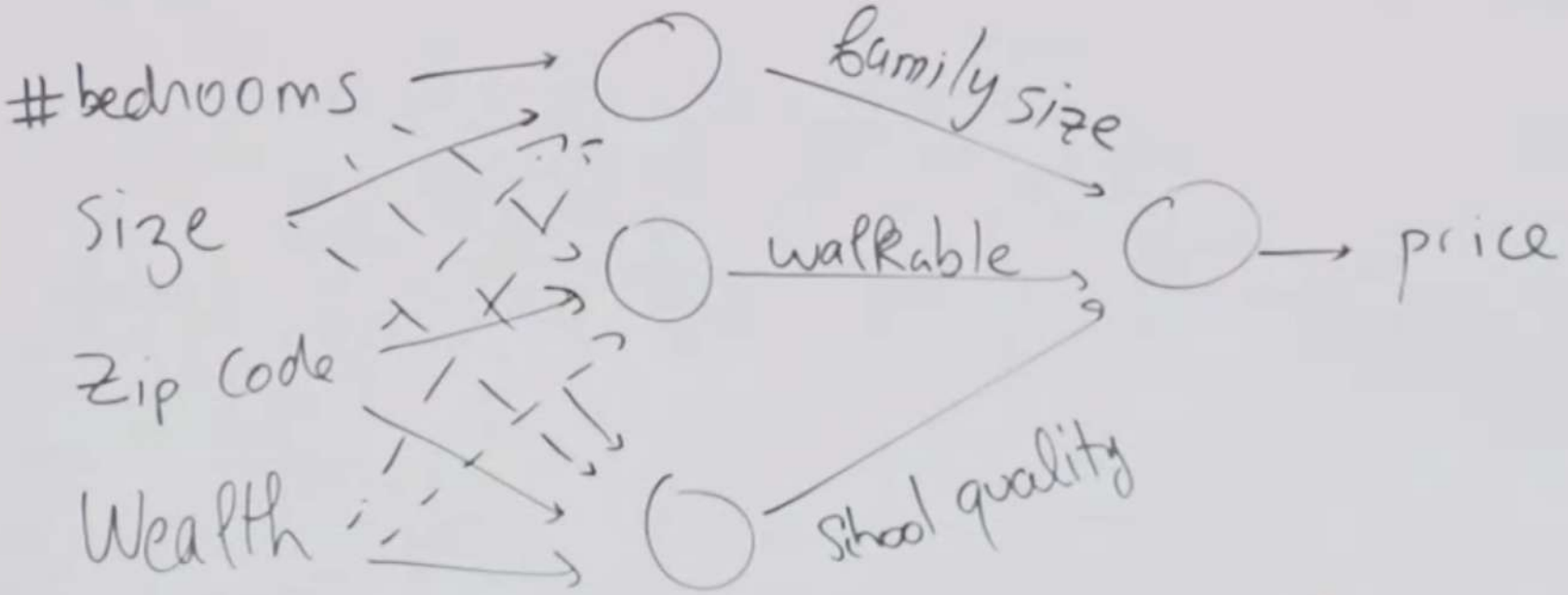


II Neural Networks

goal image ⇒ cat vs. no cat
(1) (0)



House price prediction



Neural Networks

Deep Learning

These slides have been extensively borrowed from the Dr. Sohaib's slides (Computer Vision course at LUMS, 2017).

The human brain has approximately **86 billion neurons** 🧠.

A few useful facts:

- **Neurons:** ~86 billion
- **Glial cells:** roughly comparable in number, although the exact ratio varies by brain region and measurement method
- **Synapses:** estimates are around **100–500 trillion** connections between neurons
- **Cerebral cortex:** contains roughly **16 billion neurons**
- **Cerebellum:** contains roughly **69 billion neurons**, despite being much smaller in volume

An important point related to your previous EEG question: **an EEG electrode does not record the activity of a single neuron**. It measures the combined electrical activity of **large populations of neurons**, primarily synchronized activity from cortical neurons.

🧠 How EEG works

Upgr

When neurons in the brain communicate, they generate tiny electrical currents. An **EEG system** places multiple electrodes on the scalp to detect the combined electrical activity of large groups of neurons.

The resulting signal looks roughly like a continuously changing waveform:

Time →

```
~~~~~~/\~~~~\__/~~~~/\~~~~~~~
```

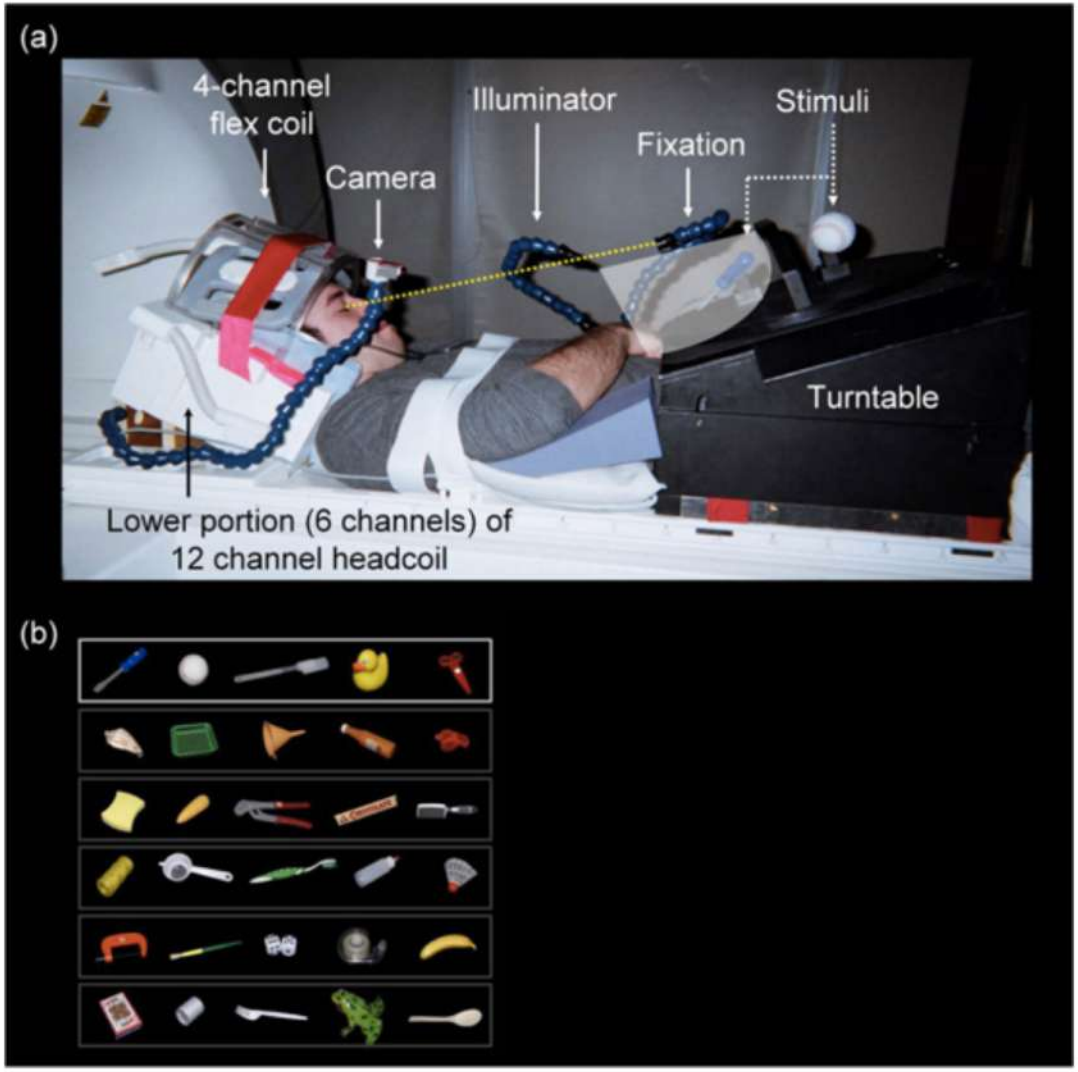
EEG typically measures voltage differences in **microvolts (µV)** over time.

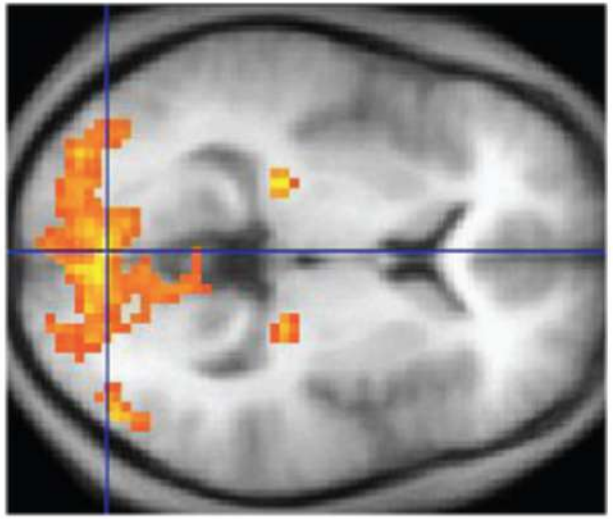
Common EEG frequency bands

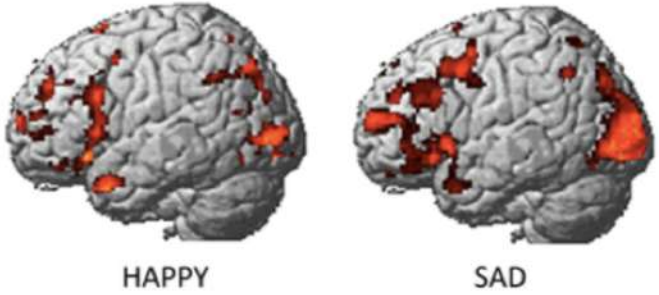
Band	Frequency	Often associated with
Delta	~0.5–4 Hz	Deep sleep
Theta	~4–8 Hz	Drowsiness, memory, relaxation
Alpha	~8–13 Hz	Relaxed wakefulness, especially eyes closed
Beta	~13–30 Hz	Alertness, active thinking
Gamma	~30–100+ Hz	Higher-frequency neural activity, perception/cognition

These associations are **not one-to-one**—you generally cannot infer a person's mental state simply from one frequency band.

Functional MRI (fMRI)

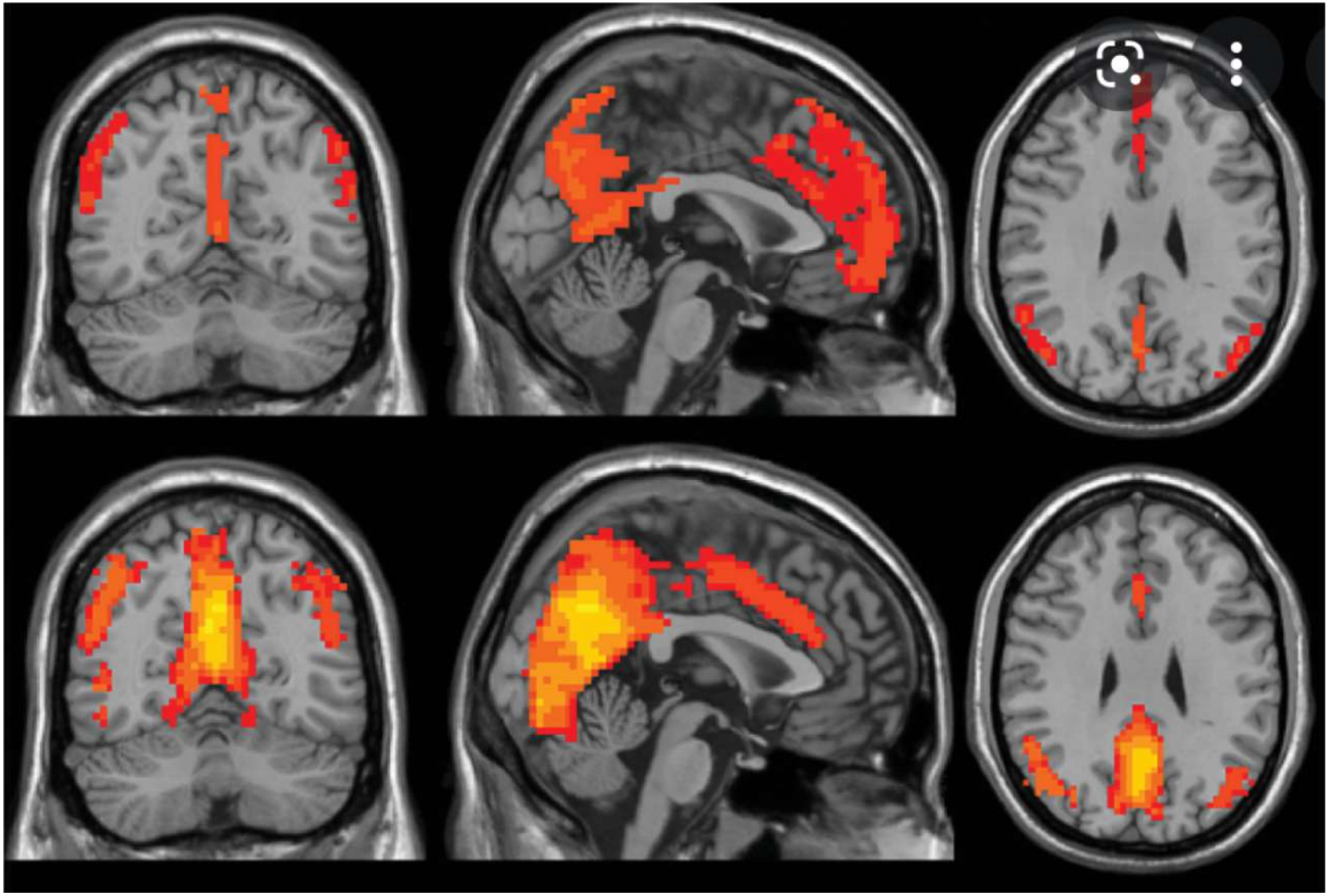




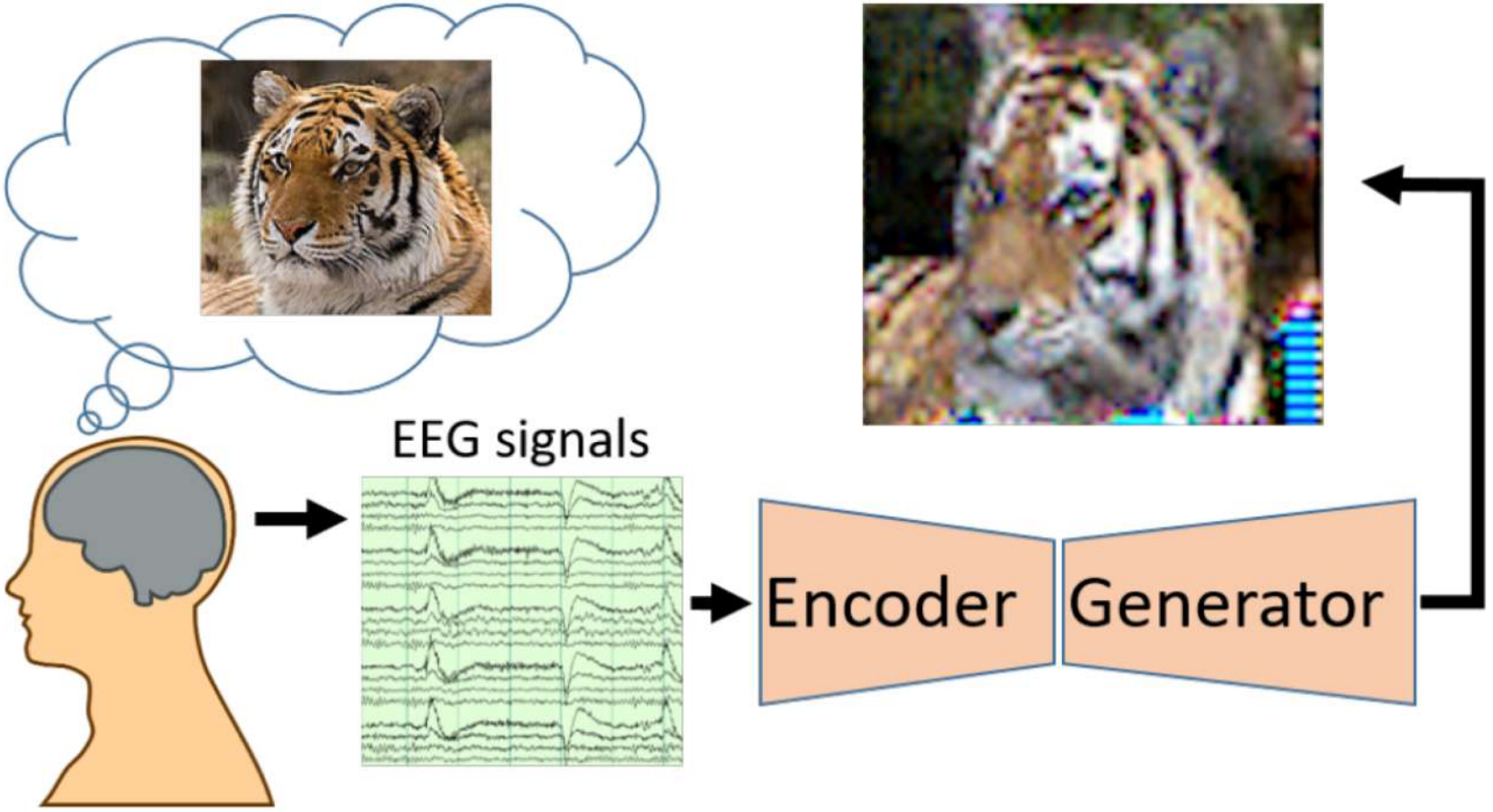


Active Regions

Functional MRI (fMRI)



EEG signals (Electroencephalography signals)

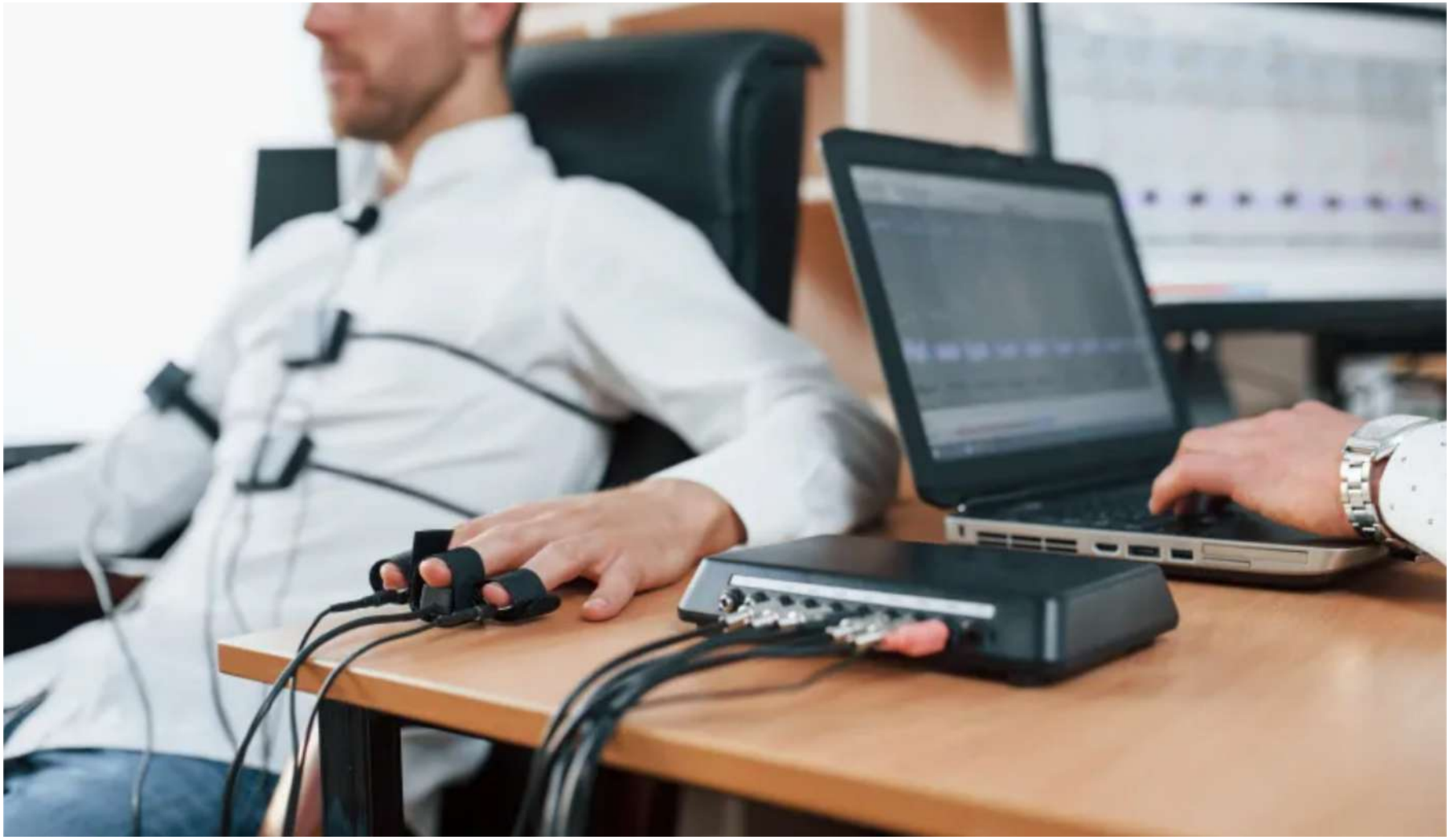


ThoughtViz: Visualizing Human Thoughts Using Generative Adversarial Network

https://www.crcv.ucf.edu/papers/acmmm18/thoughtviz.pdf

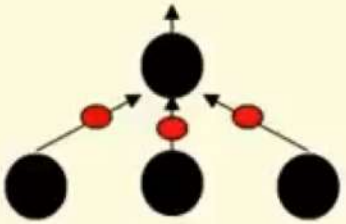
How do lie detectors work

How do lie detectors work



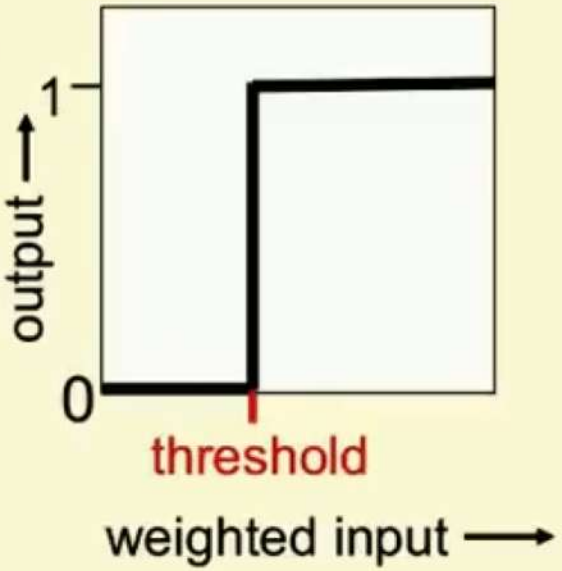
How the brain works on one slide!

- Each neuron receives inputs from other neurons
  - A few neurons also connect to receptors.
  - Cortical neurons use spikes to communicate.
- The effect of each input line on the neuron is controlled by a synaptic weight
  - The weights can be positive or negative.
- The synaptic weights adapt so that the whole network learns to perform useful computations
  - Recognizing objects, understanding language, making plans, controlling the body.
- You have about 10^{11} neurons each with about 10^{4} weights.
  - A huge number of weights can affect the computation in a very short time. Much better bandwidth than a workstation.



Binary threshold neurons

- McCulloch-Pitts (1943): influenced Von Neumann.
  - First compute a weighted sum of the inputs.
  - Then send out a fixed size spike of activity if the weighted sum exceeds a threshold.
  - McCulloch and Pitts thought that each spike is like the truth value of a proposition and each neuron combines truth values to compute the truth value of another proposition!



Binary threshold neurons

- There are two equivalent ways to write the equations for a binary threshold neuron:

$$z = \sum_i x_i w_i \qquad z = b + \sum_i x_i w_i$$

$$y = \begin{cases} 1 \text{ if } z \geq \theta \\ 0 \text{ otherwise} \end{cases}$$

Perceptron

A perceptron takes several inputs, gives each input a **weight**, adds them together, and then makes a binary decision.

$$z = w_1 x_1 + w_2 x_2 + \cdots + w_n x_n + b$$

Then an activation function determines the output:

$$y = \begin{cases} 1 & \text{if } z > 0 \\ 0 & \text{otherwise} \end{cases}$$

So the overall process is:

Inputs → Weighted Sum → Activation → Output

Example

Suppose we want to determine whether a student **passes or fails** based on:

- x_1 = study hours
- x_2 = attendance

The perceptron might learn:

$$z = 0.8x_1 + 0.5x_2 - 5$$

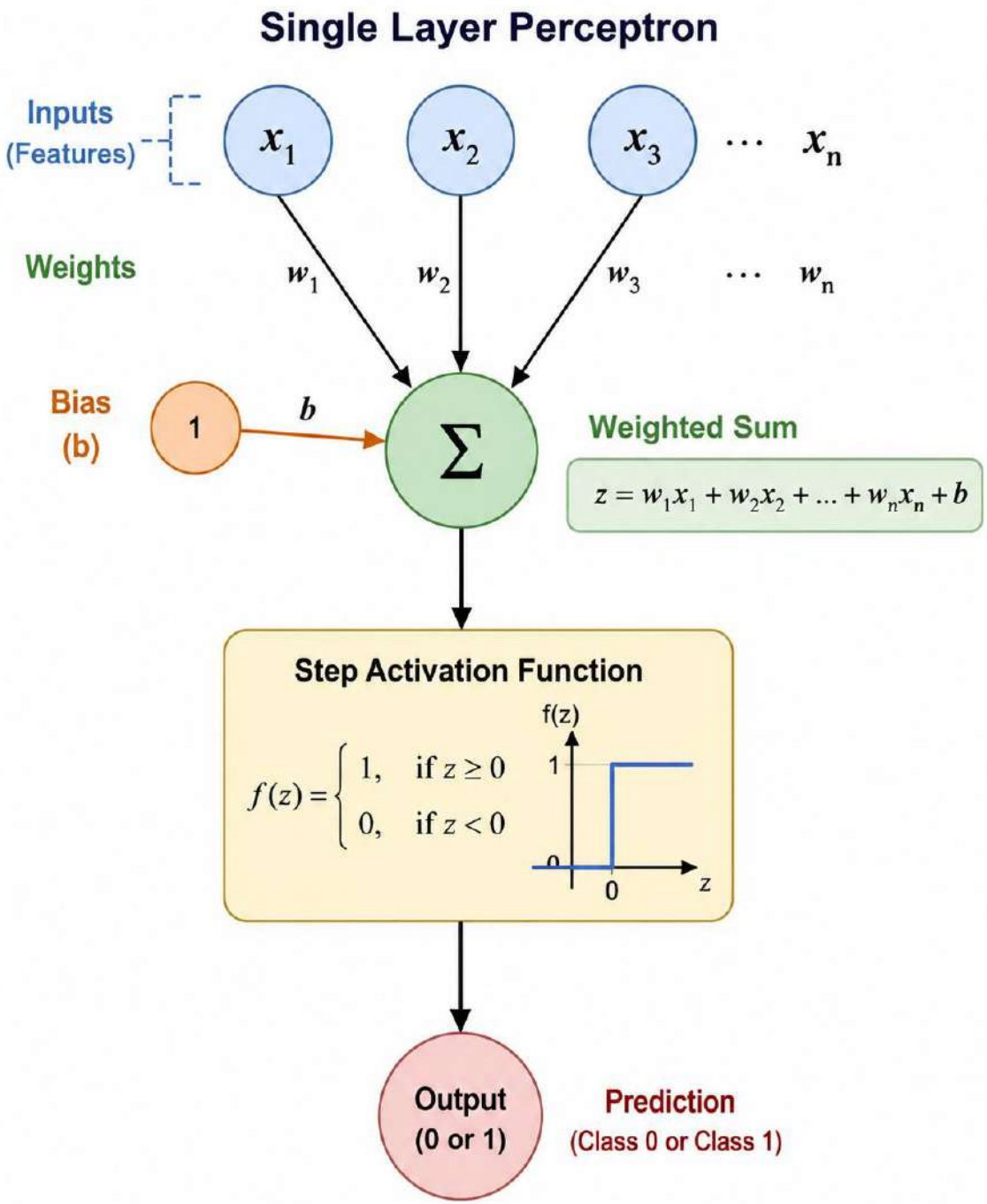
If $z > 0$:

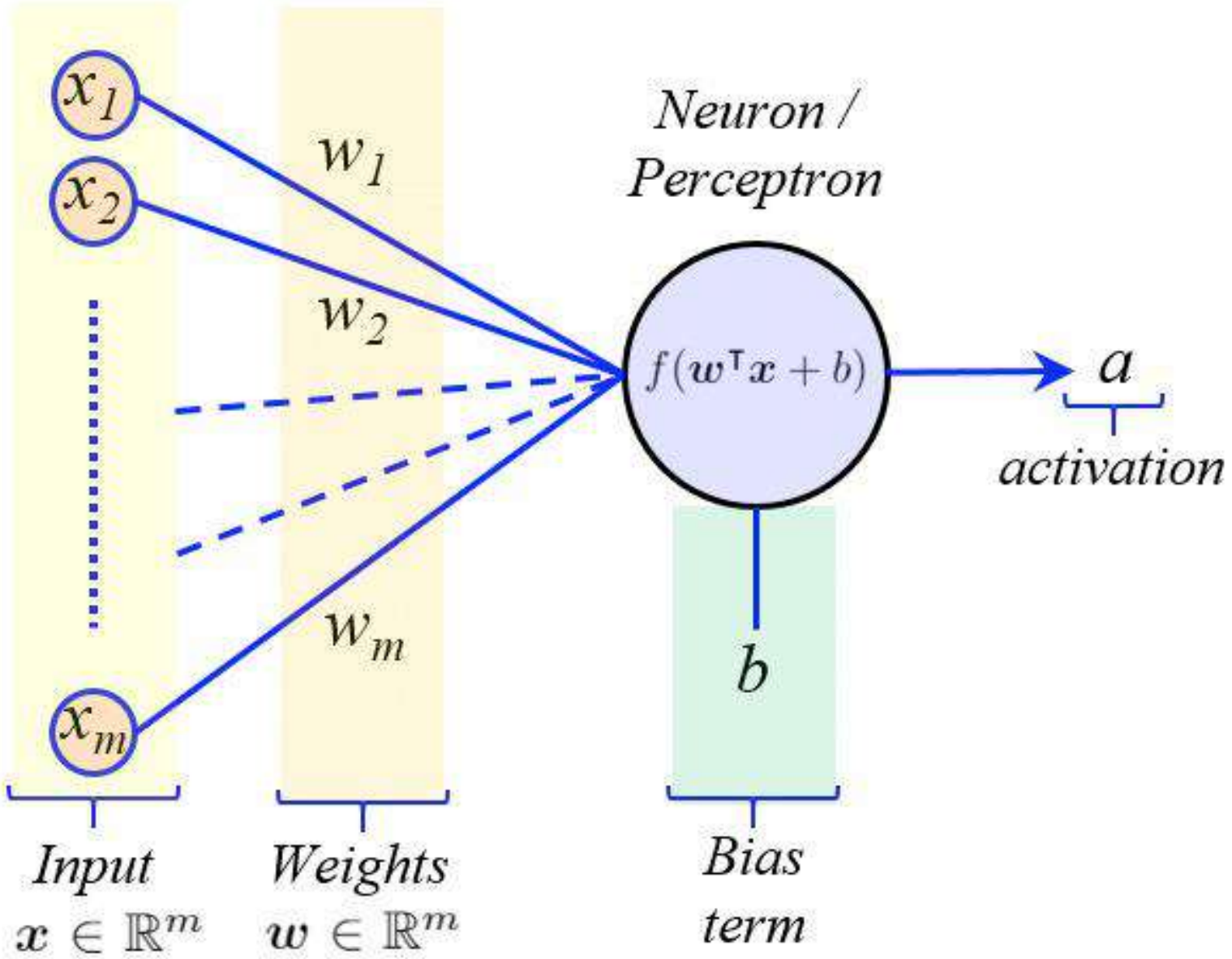
Pass = 1

Otherwise:

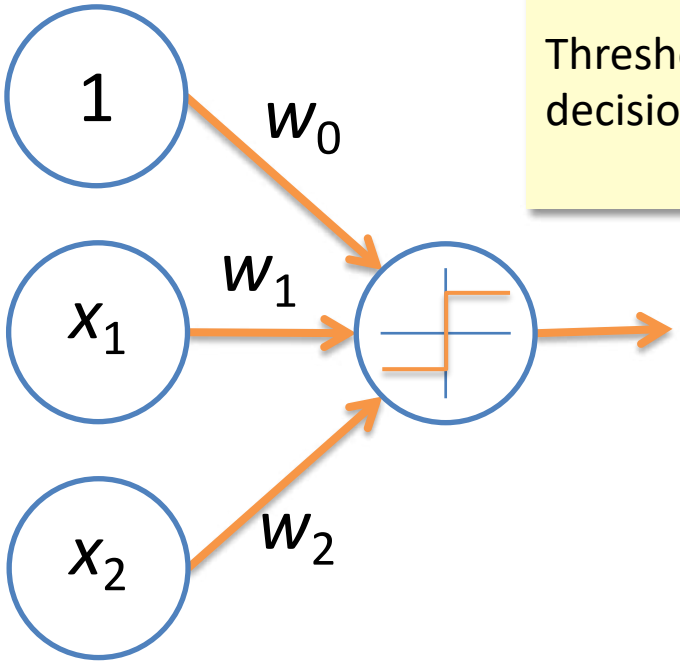
Fail = 0

The **weights** determine how important each input is, while the **bias** shifts the decision boundary.


Single Layer Perceptron
Inputs
(Features)
x_1
x_2
x_3
$\cdots$
x_n
Weights
w_1
w_2
w_3
$\cdots$
w_n
Bias
(b)
1
b
Σ
Weighted Sum
$z = w_1x_1 + w_2x_2 + \ldots + w_nx_n + b$
Step Activation Function
$f(z) = \begin{cases} 1, & \text{if } z \geq 0 \\ 0, & \text{if } z < 0 \end{cases}$
f(z)
1
0
z
Output
(0 or 1)
Prediction
(Class 0 or Class 1)


x_1
x_2
x_m
w_1
w_2
w_m
Neuron / Perceptron
$f(\boldsymbol{w}^\intercal \boldsymbol{x} + b)$
a
activation
b
Input
$\boldsymbol{x} \in \mathbb{R}^m$
Weights
$\boldsymbol{w} \in \mathbb{R}^m$
Bias term

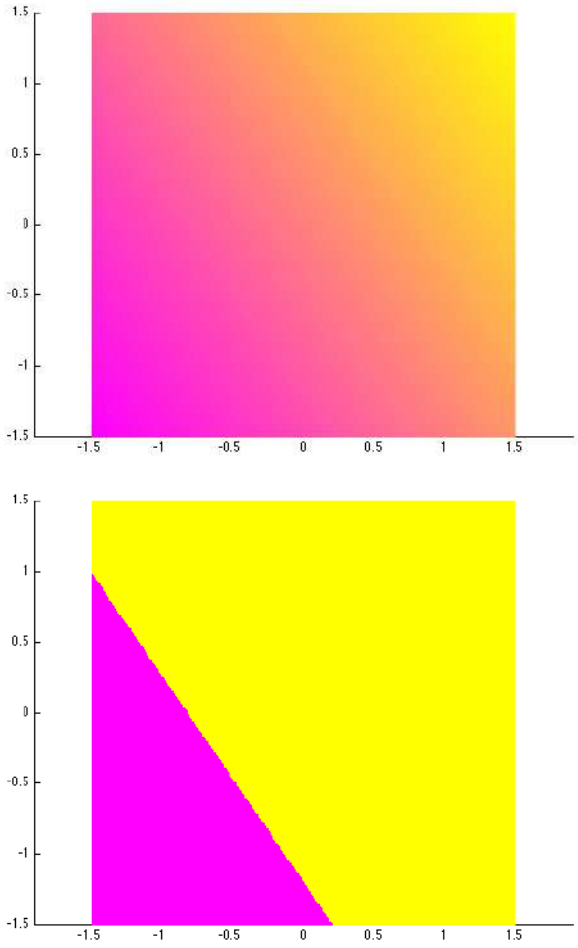
Perceptron



Implements a linear function $f(x_1, x_2)$

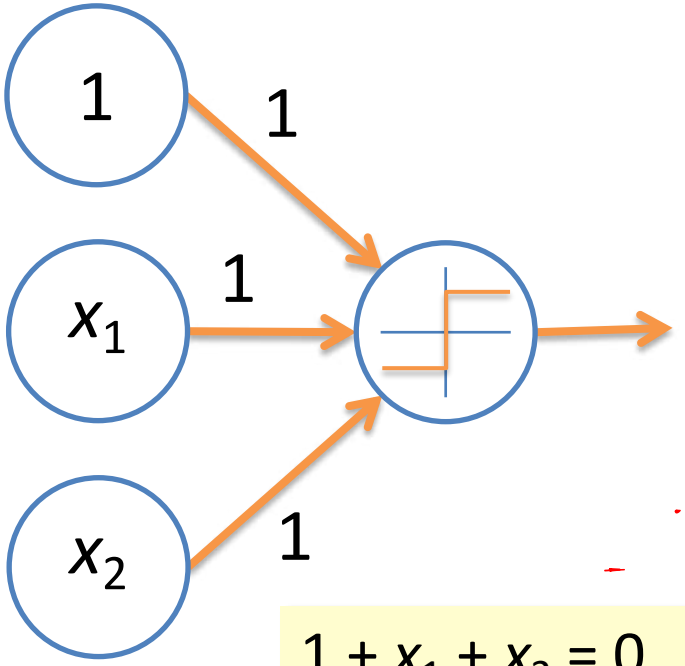
Threshold to get linear decision boundary

Example:
w = [0.79, 0.96, 0.66]

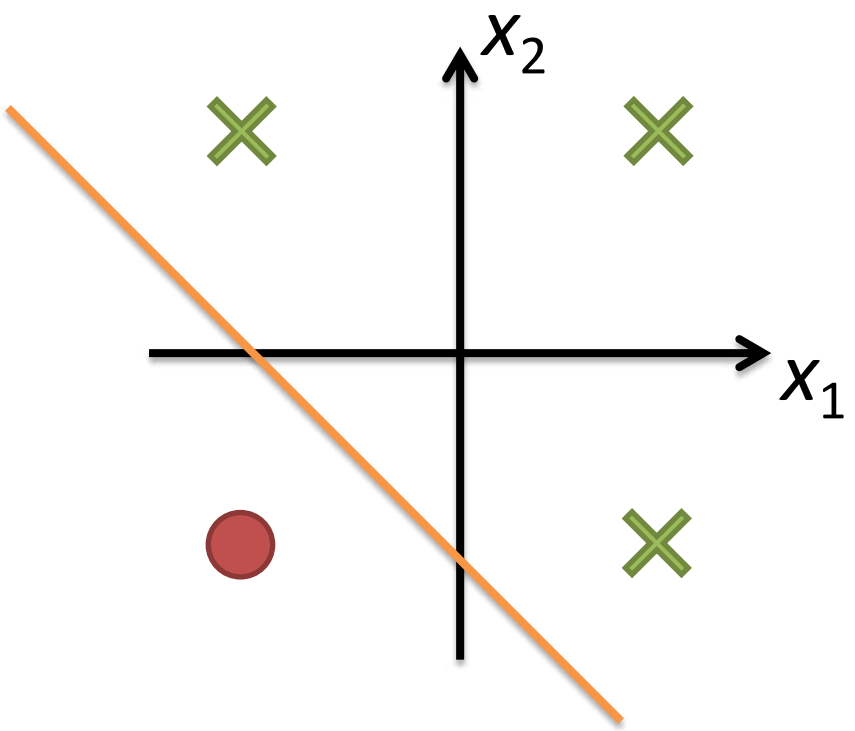


OR Function

x_1	x_2	OR(x_1, x_2)
-1	-1	-1
-1	1	1
1	-1	1
1	1	1



$1 + x_1 + x_2 = 0$

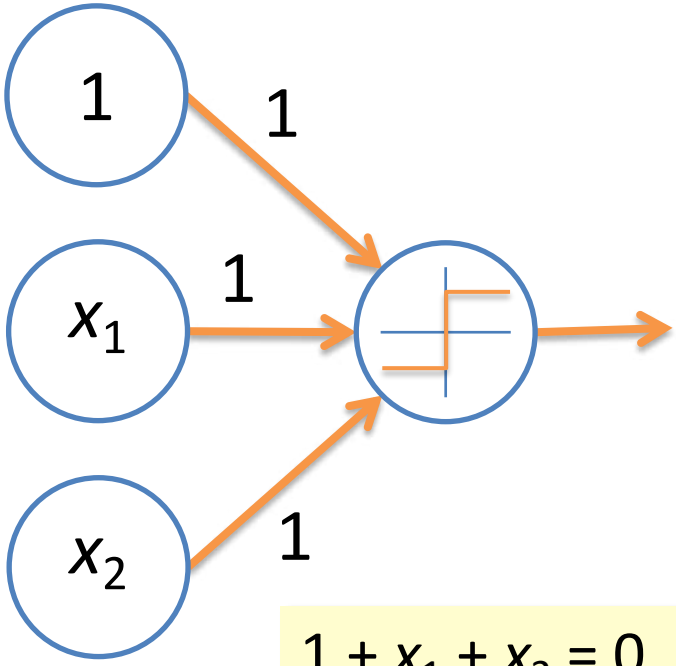


OR Function

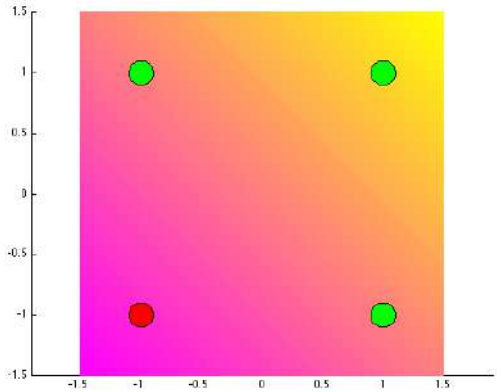
$1 \quad (w_0 + w_1 x_1 + w_2 x_2) \geq 0$

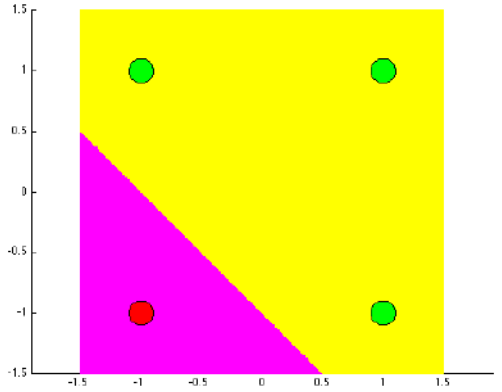
$-1 \quad$ else

x_1	x_2	OR(x_1, x_2)
-1	-1	-1
-1	1	1
1	-1	1
1	1	1



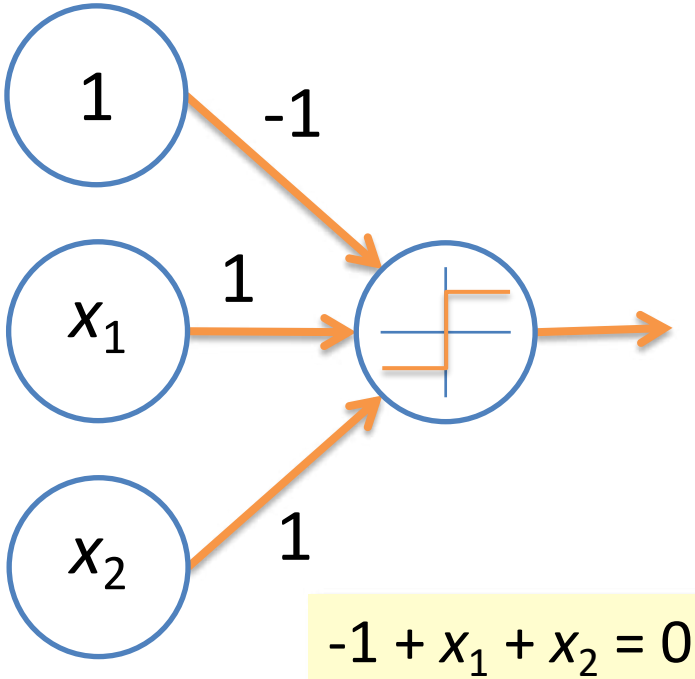
$1 + x_1 + x_2 = 0$



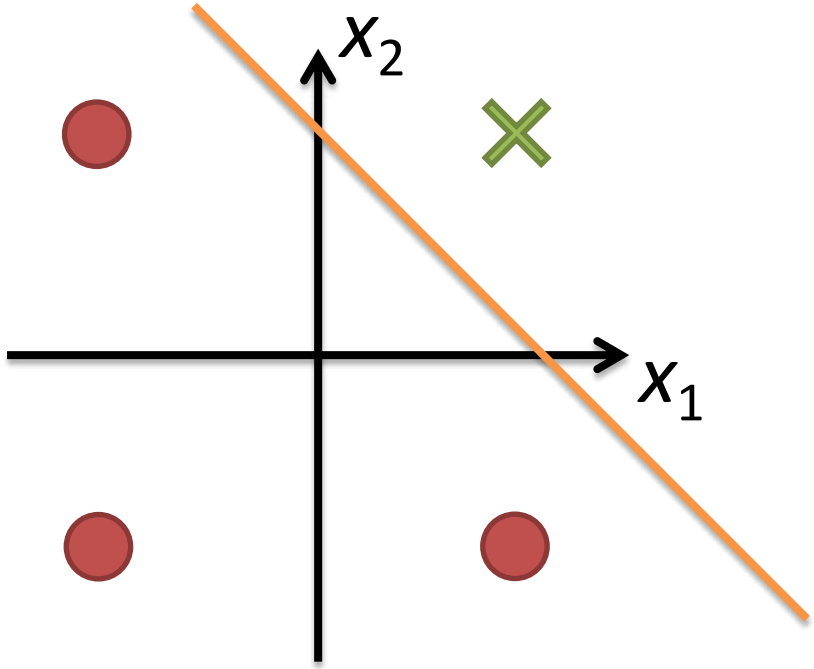


AND Function

x_1	x_2	AND(x_1, x_2)
-1	-1	-1
-1	1	-1
1	-1	-1
1	1	1

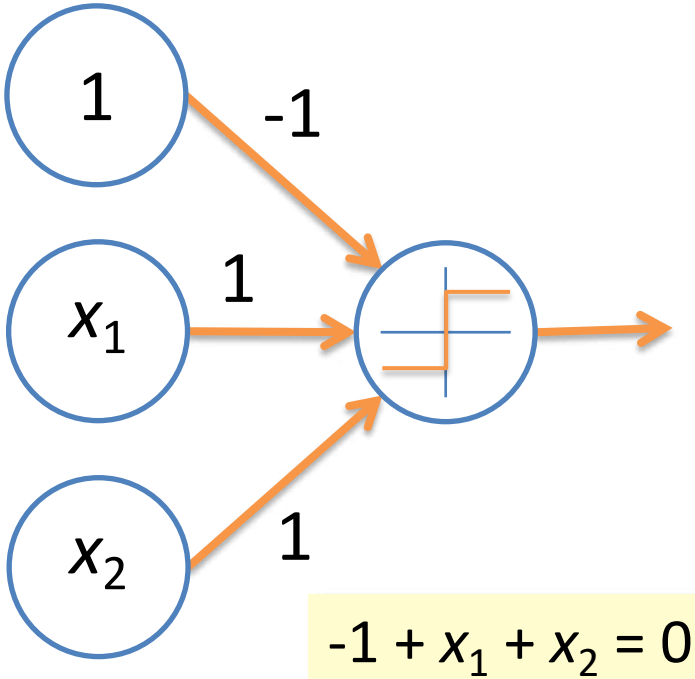


$-1 + x_1 + x_2 = 0$

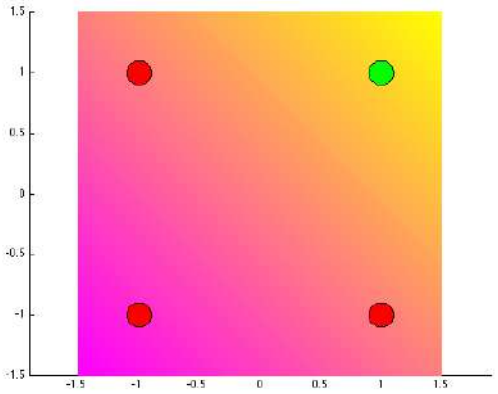


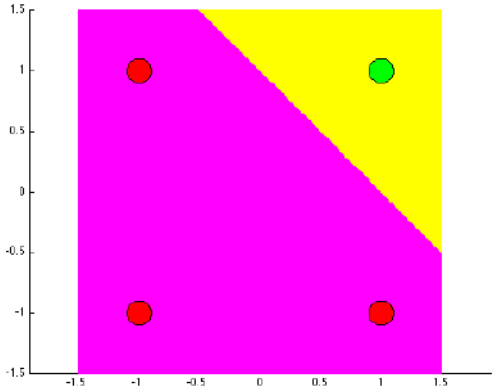
AND Functi

x_1	x_2	AND(x_1, x_2)
-1	-1	-1
-1	1	-1
1	-1	-1
1	1	1



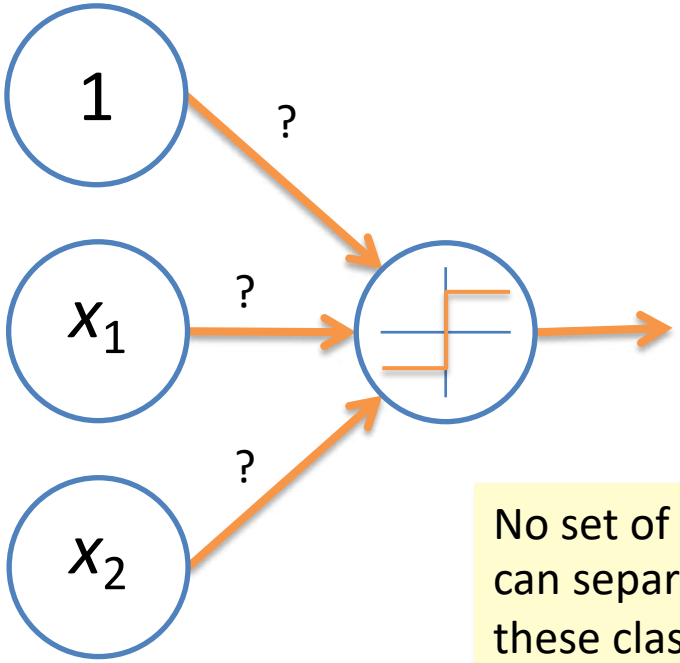
$-1 + x_1 + x_2 = 0$



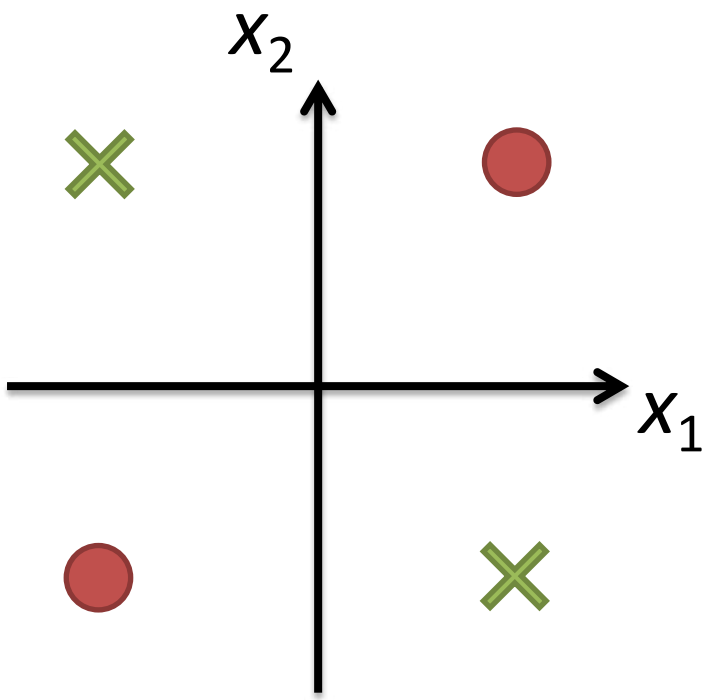


Consider XO

x_1	x_2	XOR(x_1, x_2)
-1	-1	-1
-1	1	1
1	-1	1
1	1	-1



No set of weights can separate these classes

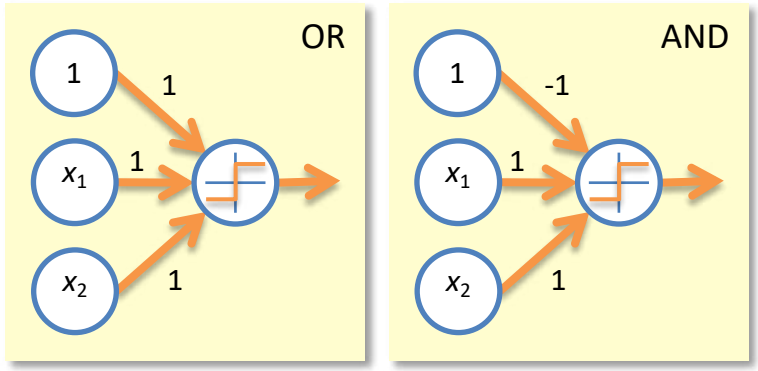


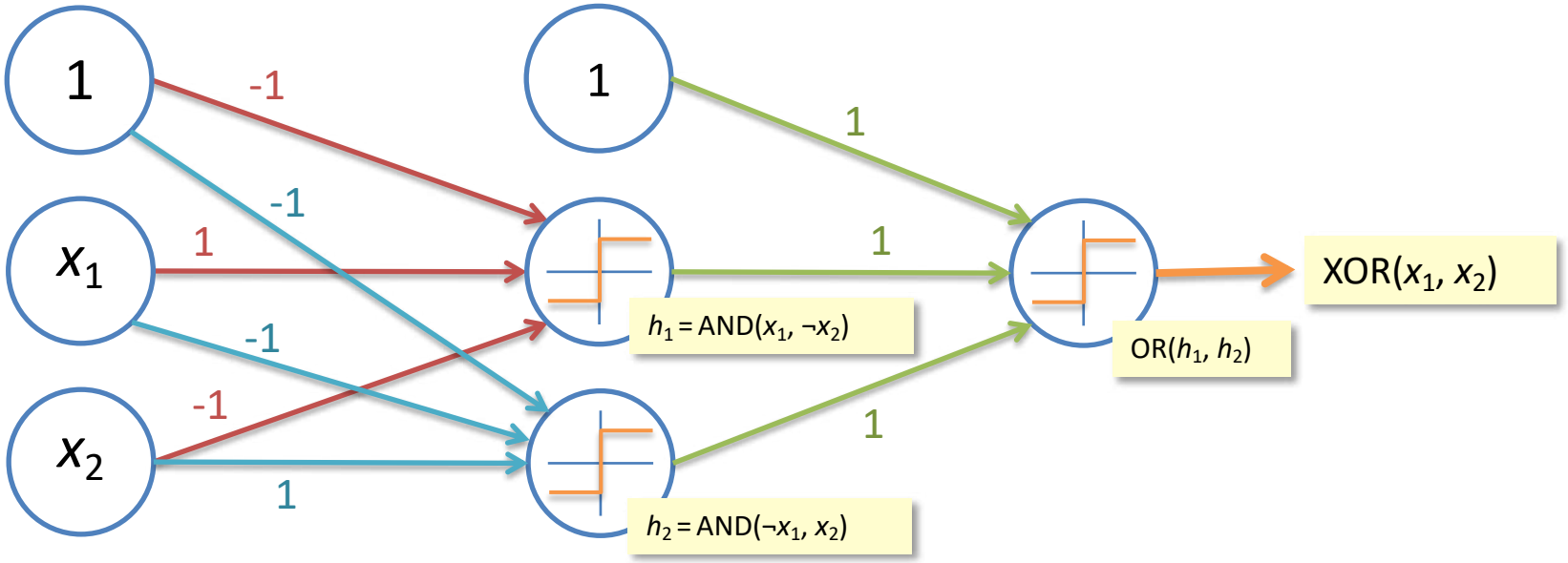
XOR

$$\text{XOR}(x_1, x_2) = \text{OR}(\text{AND}(x_1, \neg x_2), \text{AND}(\neg x_1, x_2))$$

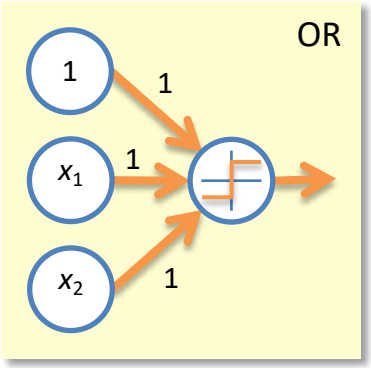
x_1	x_2	AND(x_1, $\neg x_2$)	AND($\neg x_1$, x_2)	OR(AND(x_1, $\neg x_2$), AND($\neg x_1$, x_2))
-1	-1	-1	-1	-1
-1	1	-1	1	1
1	-1	1	-1	1
1	1	-1	-1	-1

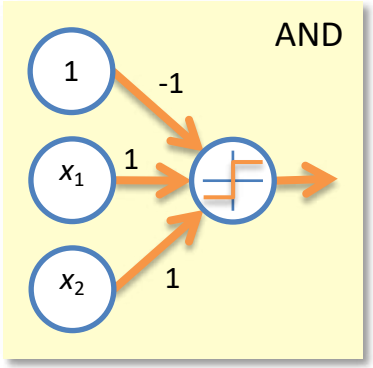
$$\text{XOR}(x_1, x_2) = \text{OR}(\text{AND}(x_1, \neg x_2), \text{AND}(\neg x_1, x_2))$$

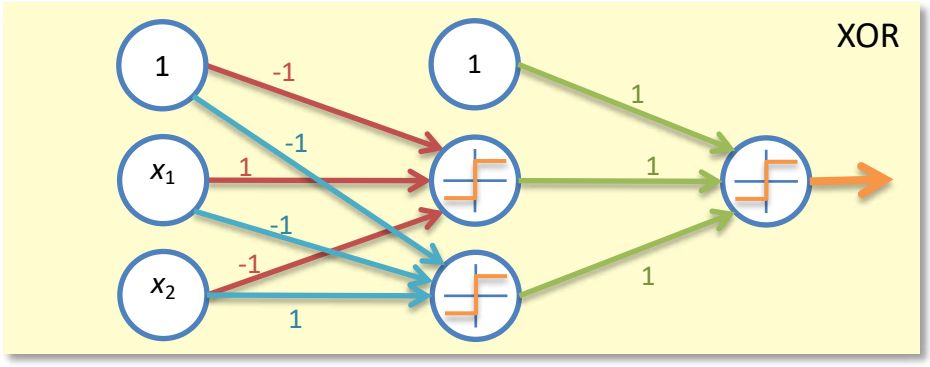




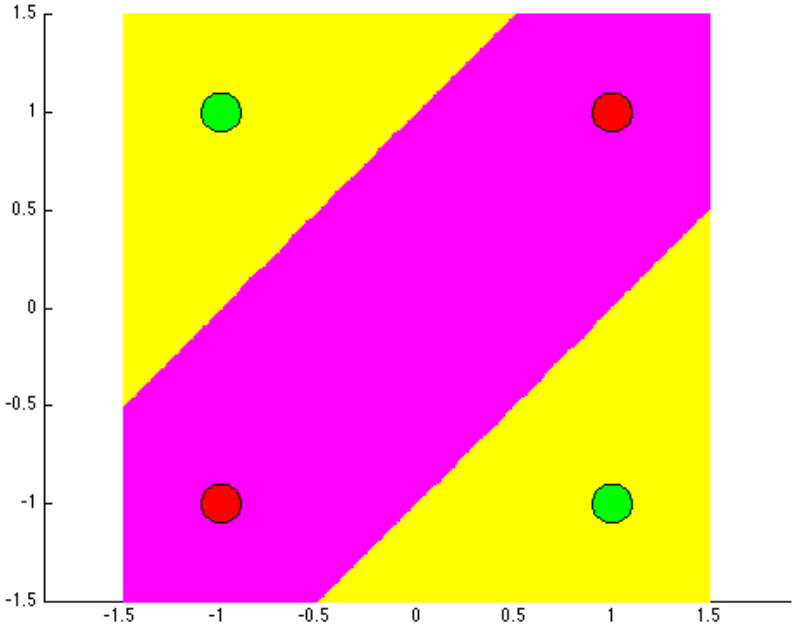
$$\mathrm{XOR}(x_1, x_2) = \mathrm{OR}(\mathrm{AND}(x_1, \neg x_2), \mathrm{AND}(\neg x_1, x_2))$$



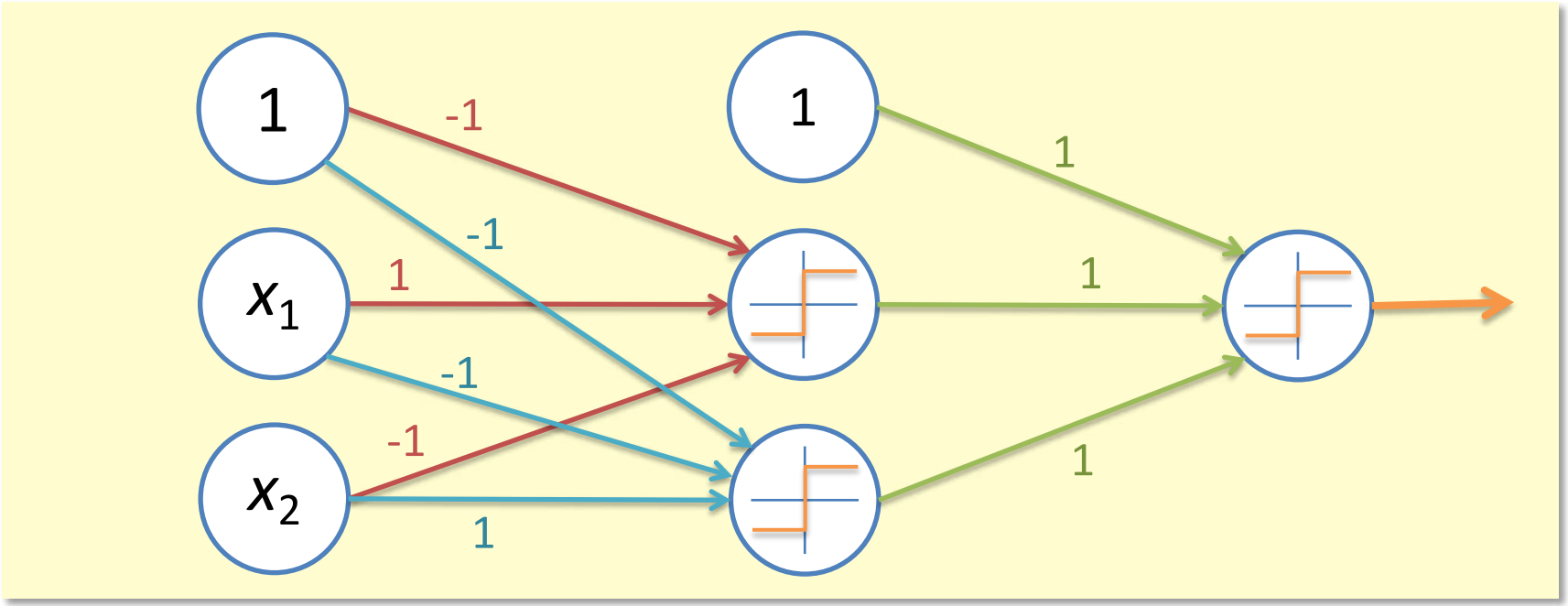




By combining two Perceptrons, we are able to create a **non-linear** decision boundary



Neural Network



"Feed-forward" – no loops allowed